

Quantization-Aware Design Space Exploration of Fixed-Point Reciprocal Approximation Architectures on Artix-7 FPGAs

NAVANEETH KUTTY*, ANISH MURALI*, and PRIYAN*, Department of Electronics & Communication Engineering, SSN College of Engineering, India

Fixed-point reciprocal computation is an important operation in field-programmable gate array (FPGA) arithmetic pipelines, where division is often replaced by dedicated approximation methods to reduce computational cost. This work presents a comparative evaluation of five reciprocal approximation architectures: Direct Lookup Table (LUT), LUT with Linear Interpolation, Newton-Raphson iteration, Goldschmidt iteration, and a hybrid LUT-Interpolation/Newton-Raphson design combining linear interpolation with a single Newton-Raphson refinement iteration, implemented in Q16.16 and Q8.24 fixed-point formats for a Xilinx Artix-7 FPGA (Nexys A7-100T) over the input range $[0.5, 2.5]$ using one million verification vectors. A Pareto analysis based on maximum relative error, latency, and energy per operation is used to characterize the trade-offs among the evaluated architectures. In Q8.24, the Direct LUT provides the lowest latency (12.9 ns) and energy per operation (0.346 nJ/op), with a maximum relative error of 0.098%, while Newton-Raphson provides the lowest error ($1.5 \times 10^{-5}\%$) at a latency of 178.0 ns and an energy consumption of 1.190 nJ/op. Goldschmidt achieves comparable numerical accuracy with a 22% reduction in latency relative to Newton-Raphson at a marginally higher energy consumption, while the Hybrid architecture provides an intermediate trade-off between numerical accuracy, latency, and energy consumption. The resulting post-implementation FPGA results provide a unified reference for evaluating fixed-point reciprocal architectures under different precision, latency, and energy constraints, with relevance to FPGA-based signal processing, normalization, coordinate transformation, and other division-free arithmetic pipelines.

CCS Concepts: • **Hardware** → *Hardware accelerators*; **Arithmetic and datapath circuits**; **Reconfigurable logic applications**.

Additional Key Words and Phrases: Reciprocal approximation, FPGA, Fixed-point arithmetic, Quantization, Newton-Raphson method, Goldschmidt algorithm, Interpolation, Pareto Analysis

ACM Reference Format:

Navaneeth Kutty, Anish Murali, and Priyan. 2018. Quantization-Aware Design Space Exploration of Fixed-Point Reciprocal Approximation Architectures on Artix-7 FPGAs. 1, 1 (October 2018), 36 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Division and reciprocal computation are comparatively expensive arithmetic operations to implement efficiently on field-programmable gate arrays (FPGAs). Unlike addition and multiplication, which map efficiently onto dedicated FPGA arithmetic resources, division typically has no equivalent dedicated hardware unit for efficient acceleration. Consequently, hardware architectures often avoid direct division or replace it with reciprocal approximation followed

*All authors contributed equally to this research.

Authors' Contact Information: Navaneeth Kutty, navaneeth2310165@ssn.edu.in; Anish Murali, anish2310158@ssn.edu.in; Priyan, priyan2310182@ssn.edu.in, Department of Electronics & Communication Engineering, SSN College of Engineering, Chennai, Tamil Nadu, India.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

by multiplication [28]. Efficient reciprocal computation is therefore important in applications where division occurs frequently or lies within an iterative processing path.

Beyond direct division, digit-recurrence, table-lookup, and functional-iteration methods together form the standard toolkit for hardware division and reciprocal computation [10]. Three broad families of reciprocal approximation are commonly considered in hardware implementations. The first relies on direct table lookup, in which the reciprocal of a quantized input is pre-computed offline and stored in on-chip memory, typically block RAM (BRAM). The second improves upon direct lookup by combining a coarser table with a linear or higher-order interpolation stage, trading additional arithmetic logic for reduced table size and approximation error. The third uses iterative multiplicative refinement, most commonly Newton-Raphson or Goldschmidt iteration, seeded from a coarse initial estimate and refined over one or more pipeline stages. Lookup-based and iterative approaches, as well as combinations of the two, have been widely explored for FPGA-based reciprocal and division computation [28]. Each family occupies a different point in the design space spanned by numerical accuracy, latency, hardware resource cost, and energy consumption, and no single architecture necessarily dominates across all of these dimensions.

A recurring difficulty in comparing these architectures, both in published studies and in practical implementations, is that results are often obtained under different evaluation conditions. Differences in fixed-point format, input range, verification methodology, target device, and reported metrics can make direct architecture-to-architecture comparison difficult. A design that exhibits favorable accuracy or resource utilization under one precision or input distribution may exhibit different behavior under another. Furthermore, synthesis-only estimates may not accurately reflect the timing and resource characteristics obtained after place-and-route.

This work addresses the difficulty of making such comparisons by evaluating five reciprocal approximation architectures: Direct LUT, LUT with Linear Interpolation (LUT-Interpolation), Newton-Raphson iteration, Goldschmidt iteration, and a hybrid LUT-Interpolation/Newton-Raphson design (Hybrid). The comparison uses a common target FPGA, numerical format, input domain, verification procedure, and implementation methodology, with architecture-specific timing constraints. All five architectures are implemented in both Q8.24 and Q16.16 fixed-point formats, synthesized and implemented for a Xilinx Artix-7 FPGA (Nexys A7-100T), and validated against one million test vectors uniformly swept over the input domain $[0.5, 2.5]$. Architecture-specific clock targets are imposed during implementation, with a 2.58-ns target period for the Direct LUT, LUT with Linear Interpolation, Newton-Raphson, and Goldschmidt architectures, and a 3.333-ns target period for the Hybrid architecture. Timing closure is treated as a primary design objective. Resource, timing, and power results are derived from post-implementation analysis, while numerical accuracy is evaluated through bit-accurate register-transfer-level (RTL) simulation against a double-precision reference model. Although latency is also evaluated, greater emphasis is placed on achievable clock frequency and timing closure to provide a consistent basis for comparing the architectures.

The remainder of this paper is organized as follows. Section 2 reviews the mathematical background of fixed-point reciprocal approximation and the table-based and iterative approaches that motivate the architectures evaluated in this work. Section 3 describes the design of each of the five evaluated architectures in detail, including the hybrid design's combination of interpolation and iterative refinement. Section 4 describes the experimental methodology, covering functional verification, timing closure, and power estimation and energy derivation. Section 5 presents the experimental results across resource utilization, timing, numerical accuracy, and power and energy, along with a cross-format comparison between Q8.24 and Q16.16. Section 6 presents a three-objective Pareto trade-off analysis over numerical error, latency, and energy per operation, followed by a consolidated summary of all reported metrics.

Section 7 discusses the practical implications, applicability, and limitations of the results, and Section 8 concludes the paper. Supporting derivations and a pairwise Pareto verification are provided in the appendices.

1.1 Previous Work and Contributions

Reciprocal computation is a fundamental operation in hardware division and numerical processing. Established hardware approaches include digit-recurrence, table-lookup, functional-iteration, and hybrid architectures, as surveyed in the standard computer-arithmetic textbooks of Ercegovac and Lang [9, 10], Parhami [30], and Muller [25], and in Soderquist and Leaser’s area/performance survey of divide and square-root implementations [35]. Oberman and Flynn provide a widely used classification of these approaches and show that practical implementations often combine multiple algorithmic classes to balance latency, cycle time, and area [28]. Functional-iteration approaches have a long history in high-performance floating-point arithmetic, including early IBM implementations and subsequent formal treatments of division by functional iteration [1, 12], which motivate the Newton-Raphson and Goldschmidt formulations evaluated in this work.

Goldschmidt’s original division-by-convergence method [14] and the classical bipartite ROM reciprocal-table construction of Das Sarma and Matula [5] established, respectively, the iterative-refinement and table-compression foundations on which much of the subsequent hardware reciprocal literature builds, including the LUT-based and Goldschmidt-family architectures evaluated in this work. Subsequent table-lookup-and-addition methods generalized the bipartite idea: Schulte and Stine’s symmetric bipartite tables [34] and de Dinechin and Tisserand’s multipartite table framework [6] unify and extend bipartite table construction to a wider range of elementary functions, while Ercegovac *et al.* [11] and Beuchat and Tisserand [3] propose small-multiplier-based reciprocal, square-root, and division architectures that trade a modest increase in multiplier count for substantially smaller seed tables than a pure LUT approach. Ito *et al.* address the complementary problem of generating a more accurate initial seed for multiplicative refinement through operand modification, reducing the number of refinement iterations required for a given target precision [16]. Piñeiro and Bruguera [31] and Piñeiro *et al.* [32] further develop minimax quadratic table-based interpolation, the former for double-precision reciprocal, division, and square-root computation and the latter for single-precision approximation of reciprocal, square-root, and other elementary functions, representing a higher-order generalization of the linear interpolation stage used by the LUT-Interpolation architecture evaluated in this work. Table-based reciprocal computation reduces arithmetic complexity by storing precomputed approximations, but memory requirements increase with precision. Chen *et al.* addressed this trade-off through a compact Taylor-based lookup structure followed by two Newton-Raphson iterations for double-precision reciprocal computation [4]. Their design uses a $2^7 \times 16$ -bit ROM and requires ten clock cycles; an FPGA implementation on a Xilinx Virtex XCV1000E was also reported. Kucukkabak and Akkas similarly combined a lookup-table initial approximation with two Newton-Raphson iterations, demonstrating the established use of table-based seeding to reduce iterative computation [20].

Fixed-point implementations have subsequently explored more hardware-efficient forms of approximation and iteration. Rodríguez-García *et al.* proposed a fixed-point divider based on reciprocal computation using a six-segment second-order polynomial approximation for the initial seed and one Newton-Raphson iteration. Scaling and de-scaling allow the reciprocal calculation to operate over a normalized interval while supporting different fixed-point formats [33]. Libessart *et al.* investigated a pipelined fixed-point reciprocal operator based on Newton-Raphson, further demonstrating the importance of fixed-point representation and hardware pipeline organization [23]. Lunglmayr proposed an FPGA-oriented non-sequential reciprocal approximation based on piecewise linearization with power-of-two node points

and a polynomial correction stage, targeting scaling-free, LUT-free implementation for large data-processing pipelines rather than the BRAM-seeded architectures evaluated here [24].

FPGA-oriented comparative work is also available. Kumar, Saha, and Dandapat implemented fixed-point reciprocal and division using both digit-recurrence and multiplicative methods, including Newton-Raphson and Goldschmidt algorithms. Their designs were implemented in Verilog and synthesized on a Xilinx Virtex-4 FPGA, with frequency, slice utilization, latency, and power evaluated for several operand lengths [21]. Kasim *et al.* investigated another memory-oriented approach in which a precomputed reciprocal value is selected using a reduced number of denominator bits. Their results explicitly demonstrate the exponential trade-off between lookup-table size and approximation error, and the work compares the resulting divider with Goldschmidt and non-restoring implementations [18]. In the floating-point domain, Wang and Nelson compared digit-recurrence and multiplicative division and square-root architectures on Virtex FPGAs [38]. Sorokin evaluated high-speed fixed-point dividers against a vendor IP core baseline [36], and Muñoz *et al.* reported an analogous area/latency/power tradeoff study of Goldschmidt- and Newton-Raphson-based floating-point division and square-root cores [26]; the present work follows a similar comparative methodology but holds the fixed-point format, input domain, and target device fixed across five reciprocal-only architectures rather than varying operator type. Nannarelli and Lang additionally show that power and delay trade off differently across divider radices [27], reinforcing the need for the explicit power and energy characterization performed in Section 5.4 of this work.

Goldschmidt-based fixed-point designs have also been studied specifically for speed and energy efficiency. Kong and Swartzlander proposed a modified Goldschmidt update that achieves faster-than-quadratic (near-cubic) convergence by reusing an intermediate squaring result, reducing the number of refinement iterations needed for a given target precision relative to the standard quadratically convergent formulation used in this work [19]. Ercegovic *et al.* similarly improve Goldschmidt division, square-root, and square-root-reciprocal computation by replacing the final iteration with a looked-up correcting term [8], while Gallagher and Swartzlander address fault tolerance in time-shared triple-modular-redundant Newton-Raphson and Goldschmidt dividers [13]. Paim *et al.* proposed a modified Goldschmidt divider using a suitable denominator approximation and evaluated its convergence and energy characteristics for Q7.8 arithmetic [29]. More recently, Gutiérrez-Ramos *et al.* presented a multifunctional fixed-point architecture combining piecewise-polynomial seed generation with Newton-Raphson iterations for reciprocal, division, square-root, and reciprocal-square-root operations [15]. Their verified hardware configuration uses unsigned Q(16,5,u) and was synthesized on an Intel Stratix-V FPGA.

These studies establish the individual techniques considered in this work, but their implementation conditions vary substantially. Differences in numerical format, precision, input normalization, FPGA family, synthesis tools, architecture, and reported metrics make direct cross-study comparison difficult. Existing comparative work is valuable but generally evaluates a narrower set of methods or focuses on a particular implementation objective rather than examining the combined effects of reciprocal algorithm and fixed-point precision within one common FPGA framework.

1.2 Contributions of This Work

This work presents a systematic post-implementation hardware evaluation of five established reciprocal architectures: **Direct LUT, LUT-Interpolation, Newton-Raphson, Goldschmidt, and Hybrid LUT-Interpolation/Newton-Raphson approach**. Each architecture is implemented in both **Q8.24** and **Q16.16** fixed-point formats, resulting in ten architecture/format configurations evaluated over a common reciprocal input domain.

All implementations are evaluated on the **Xilinx Artix-7 XC7A100T** using architecture-specific clock targets selected according to the timing requirements of each datapath. The evaluation considers **numerical accuracy, FPGA resource utilization, timing closure, latency, throughput, post-implementation power estimates, and energy per operation**. Timing closure is assessed independently for each architecture/format combination using post-place-and-route timing analysis. All ten configurations achieve **positive worst negative slack (WNS) at their respective target periods**, providing a consistent basis for comparing the implementation effects of reciprocal architecture and fixed-point precision.

The hardware implementations are pipelined according to their respective datapath structures, with **BRAM, slice LUT/flip-flop (FF), and digital signal processing (DSP) resources** utilized according to the requirements of each architecture. The resulting pipeline depths and achieved clock frequencies are used to characterize the latency and throughput of each configuration. Particular emphasis is placed on the interaction between **arithmetic precision, pipeline structure, resource utilization, and achievable timing**, allowing the comparison to distinguish the effects of fixed-point format from those of the underlying reciprocal architecture.

The contribution is therefore **not the introduction of a new reciprocal algorithm**, but a controlled comparative study of established approaches under a common FPGA implementation environment and two fixed-point precisions. By evaluating **accuracy, resource utilization, timing, latency, power, and energy** under a common target device, numerical format, input domain, and verification procedure, the study establishes quantitative design points spanning **memory-based approximation, interpolation-based approximation, iterative reciprocal computation, and hybrid architectures**. The resulting comparison provides a common hardware-level basis for examining the trade-offs among these approaches and for understanding how fixed-point precision influences their implementation characteristics.

2 Background

2.1 Fixed-Point Reciprocal Computation

A signed fixed-point number in $QI.F$ format represents a real value v using I integer bits and F fractional bits. Given a stored two's-complement integer representation r , the represented value is

$$v = r \times 2^{-F} \quad (1)$$

where the exponent $-F$ corresponds to the implicit binary point positioned F bits from the least-significant bit (LSB). This work evaluates two 32-bit fixed-point configurations: Q8.24, with 8 integer bits and 24 fractional bits, and Q16.16, with 16 integer bits and 16 fractional bits. Q8.24 provides greater fractional precision at the expense of dynamic range, whereas Q16.16 provides a larger integer range with lower fractional precision.

The reciprocal computation considered in this work takes a fixed-point input $x \in [0.5, 2.5]$ and produces a fixed-point output y approximating $1/x$. The bounded input range provides a common operating domain for all evaluated architectures. In a complete reciprocal datapath, arbitrary positive inputs can be range-reduced using leading-bit or exponent normalization before reciprocal evaluation, followed by appropriate re-scaling. This work focuses on the reciprocal evaluation stage and does not include the preceding normalization or subsequent re-scaling logic.

2.2 LUT-Based Approximation

The most direct approach to reciprocal approximation is to precompute $1/x$ for a quantized set of input values and store the results in a lookup table, typically implemented using BRAM. For an N -bit address, a direct LUT requires 2^N entries, with each entry having sufficient width to represent the reciprocal at the desired output precision. The main advantage of this approach is its simple datapath and low latency after pipeline fill. However, increasing the address width causes the memory requirement to grow exponentially, while reducing the address width increases the input quantization error.

LUT-based accuracy can be improved without a proportional increase in table size by adding a linear interpolation stage. Instead of storing the reciprocal at every quantized input value, the table stores reciprocal values at a smaller number of breakpoints, and the value between adjacent breakpoints is estimated using the local slope:

$$y(x) = L[i] + (x - x_i) \times S[i] \quad (2)$$

where x_i is the breakpoint immediately below x , $L[i]$ is the stored reciprocal value at that breakpoint, and $S[i]$ is the slope between adjacent breakpoints. The interpolation stage therefore reduces the required table size at the cost of additional arithmetic logic.

2.3 Iterative Reciprocal Approximation

Iterative methods use a coarse initial estimate followed by one or more refinement stages to improve the reciprocal estimate. Two iterative approaches are considered in this work: Newton-Raphson and Goldschmidt iteration.

Newton-Raphson iteration for the reciprocal function can be derived by finding the root of $f(y) = 1/y - x$. Using the standard Newton update, $y_{n+1} = y_n - f(y_n)/f'(y_n)$, with $f'(y) = -1/y^2$ gives

$$y_{n+1} = y_n \times (2 - x \times y_n) \quad (3)$$

The method converges quadratically when the initial estimate y_0 is sufficiently close to $1/x$. In this work, y_0 is generated using an 8-bit-address seed LUT and followed by two Newton-Raphson refinement iterations. Each refinement iteration contains two multiplication operations and one subtraction. Thus, the complete Newton-Raphson datapath performs four multiplications and two subtractions per reciprocal estimate.

Goldschmidt iteration uses a related multiplicative refinement process. Given an initial estimate $y_0 \approx 1/x$, the initial values are

$$N_0 = y_0, \quad D_0 = x \times y_0 \quad (4)$$

The refinement factor and subsequent updates are

$$F_i = 2 - D_i, \quad N_{i+1} = N_i \times F_i, \quad D_{i+1} = D_i \times F_i \quad (5)$$

As the iterations proceed, D_i approaches unity and N_i approaches $1/x$. The two multiplication paths can be evaluated in parallel within an iteration, providing greater hardware parallelism than the sequential multiplication structure of Newton-Raphson. This shortens the dependency chain without increasing the number of multiplications. In this work, the Goldschmidt datapath is evaluated using two refinement iterations, with the N_2 value used as the final reciprocal estimate.

2.4 Theoretical Error Analysis

The measured accuracy differences among the evaluated architectures can be related to the approximation and convergence properties of their underlying methods.

For a Direct LUT with an N -bit input address over the domain $[a, b] = [0.5, 2.5]$, the nominal input quantization interval is

$$\Delta x = \frac{b - a}{2^N}. \quad (6)$$

Since the reciprocal function $f(x) = 1/x$ has its largest magnitude derivative at the lower boundary, $|f'(x)| = 1/x^2$, $|f'(0.5)| = 4$, a first-order estimate of the maximum quantization-induced reciprocal error is

$$e_{\max, \text{LUT}} \approx |f'(0.5)| \Delta x = 4\Delta x. \quad (7)$$

For the 12-bit Direct LUT evaluated in this work, this gives $e_{\max, \text{LUT}} \approx 4 \left(\frac{2}{2^{12}} \right) = 2^{-9} \approx 1.953 \times 10^{-3}$. This estimate is close to the measured maximum absolute errors of 1.950×10^{-3} for Q8.24 and 1.892×10^{-3} for Q16.16 reported in Table ?? . The agreement indicates that input-address quantization is the dominant error mechanism for the Direct LUT over the evaluated domain, although the first-order model does not capture the complete effects of address mapping and output quantization.

For the LUT with linear interpolation, the ideal interpolation error is substantially smaller than the direct LUT quantization error. Consequently, the measured interpolation error is primarily influenced by finite-precision representation of the stored interpolation coefficients and intermediate arithmetic rather than by the ideal interpolation remainder.

For the iterative architectures, convergence can be characterized directly using the relative deviation $\epsilon_n = 1 - x \times y_n$. Substituting the Newton-Raphson update from Eq. (3) gives $x \times y_{n+1} = 2(x \times y_n) - (x \times y_n)^2$ and therefore

$$\epsilon_{n+1} = \epsilon_n^2. \quad (8)$$

This is an exact algebraic identity: each refinement iteration squares the current relative deviation. Consequently, the number of correct bits approximately doubles with each iteration while the error remains above the fixed-point quantization floor.

An equivalent error relation applies to Goldschmidt iteration. With $\epsilon_i = 1 - D_i$, the update in Eq. (5) produces the same error-squaring behavior, while the invariant relationship between N_i and D_i preserves the reciprocal estimate. Thus, both iterative methods exhibit quadratic convergence, although their hardware organizations differ. The derivations, together with a comparison of closed-form Direct LUT bounds against the measurements, are given in Appendix ?? .

Equation (8) also explains the accuracy improvement obtained by the Hybrid architecture. Its interpolated seed provides a smaller initial deviation ϵ_0 than the coarse LUT seed used by the standalone iterative architectures. The subsequent Newton-Raphson refinement then squares this smaller error, allowing the Hybrid architecture to achieve high numerical accuracy with a single refinement stage.

3 Reciprocal Architecture Design

This section describes the hardware design of each of the five reciprocal approximation architectures evaluated in this work. All five architectures share a common input format, output format, and pipelined register discipline (Section 3.6), differing only in their internal datapath. Design parameters for all five architectures, in both evaluated fixed-point formats, are summarized in Table 1. Throughout, “LUT” alone denotes a reciprocal lookup table, whereas the FPGA logic resource is called a *slice LUT*.

Table 1. Design Parameters by Architecture and Fixed-Point Format

Architecture	Format	Int/Frac Bits	Clock Target (ns)	LUT Address Bits
Direct LUT	Q8.24	8/24	2.58	12
Direct LUT	Q16.16	16/16	2.58	12
LUT-Interpolation	Q8.24	8/24	2.58	11
LUT-Interpolation	Q16.16	16/16	2.58	11
Newton-Raphson	Q8.24	8/24	2.58	8
Newton-Raphson	Q16.16	16/16	2.58	8
Goldschmidt	Q8.24	8/24	2.58	8
Goldschmidt	Q16.16	16/16	2.58	8
Hybrid	Q8.24	8/24	3.333	9
Hybrid	Q16.16	16/16	3.333	9

3.1 Direct LUT

The Direct LUT architecture quantizes the input x to the seed address width (12 bits, per Table 1) and uses the quantized address to directly index a single block-RAM-resident lookup table storing precomputed reciprocal values at full output precision. The datapath (Fig. 1) consists of an input register, an address-formation stage that extracts the top address-width bits of the bounded input representation, the BRAM read itself, and an output register. No arithmetic (multiply or add) logic is required in the data path beyond address formation, which is reflected in Table 3's near-zero slice LUT utilization for this architecture. The trade-off, visible throughout Section 5, is that Direct LUT, despite having the widest address (12 bits) of the five architectures, is a piecewise-constant approximation with no interpolation or refinement stage and consequently exhibits the largest numerical error, concentrated near the lower end of the input domain where $1/x$ changes most rapidly per unit of x .

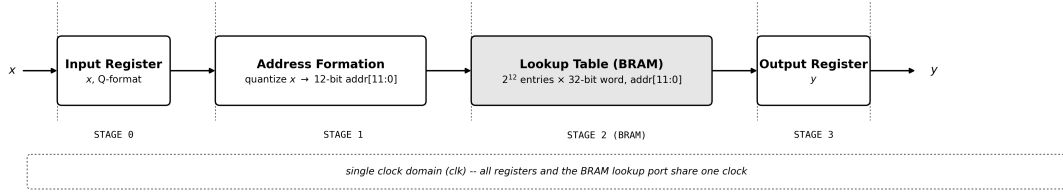


Fig. 1. Direct LUT reciprocal architecture.

3.2 LUT-Interpolation

The interpolation architecture reduces the seed table to 11 address bits and adds a linear interpolation stage per Eq. (2). In addition to the coarser reciprocal-value table, a second table (or a second field of the same table row) stores the local slope $S[i]$ between the current breakpoint and the next. The datapath (Fig. 2) computes the fractional offset $(x - x_i)$

from the low-order bits of the input not consumed by the address, multiplies this offset by the stored slope, and adds the product to the base table value $L[i]$. This requires one multiplier and one adder beyond the Direct LUT datapath, which is reflected in the non-zero DSP and substantially higher slice LUT/FF utilization reported in Table 3, but yields roughly an order-of-magnitude improvement in maximum relative error relative to the Direct LUT architecture (9.4× in Q8.24 and 10.4× in Q16.16) even though its address is only one bit narrower (11 versus 12 bits).

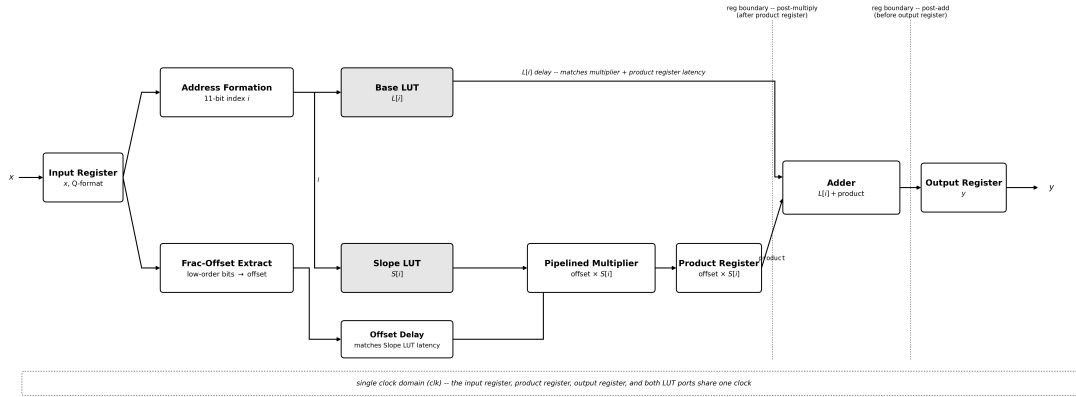


Fig. 2. LUT-Interpolation reciprocal architecture.

3.3 Newton-Raphson

The Newton-Raphson architecture computes an 8-bit-address seed estimate y_0 from a coarse BRAM lookup table and applies two Newton-Raphson refinement iterations using Eq. (3). The first iteration computes $y_1 = y_0(2 - xy_0)$, while the second computes $y_2 = y_1(2 - xy_1)$, with y_2 used as the final reciprocal estimate. Each iteration consists of two sequential 32×32 fixed-point multiplications with a subtraction from the constant 2 between them, giving four sequential multiplications and two subtraction operations per reciprocal estimate. The datapath is implemented using the deeply pipelined `pipelined_mult32x32_signed` multiplier, instantiated once per multiplication (four instances) so that a new input can be accepted every cycle, and the `reciprocal_nr_q8_24` iteration-control structure. Because the four multiplications form a serial dependency chain across the two refinement iterations, the Newton-Raphson architecture has the deepest multiplier pipeline among the evaluated architectures, resulting in a 69-cycle pipeline at the target clock (Fig. 3).

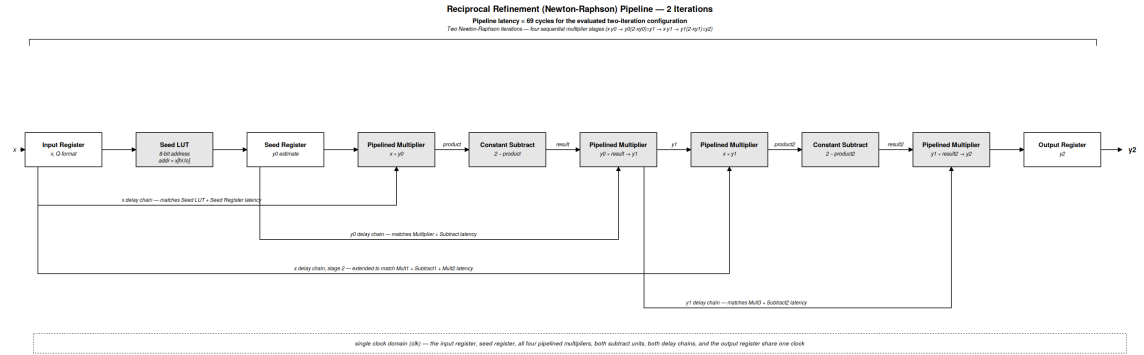


Fig. 3. Newton-Raphson reciprocal architecture.

3.4 Goldschmidt

The Goldschmidt architecture uses the same 8-bit-address seed table as Newton-Raphson but restructures the refinement as the mutually recursive numerator/denominator update of Eqs. (4) and (5). The two multiplications $F_i \times N_i$ and $F_i \times D_i$ within a given iteration do not depend on one another and can therefore be issued in the same cycle on two separate multiplier instances. Both Newton-Raphson and Goldschmidt instantiate four fully pipelined 32×32 multipliers (20 DSP48E1 slices in each design), so this parallelism does not add multiplier resources; it shortens the chain of dependent multiplications from four stages to three, and hence the pipeline from 69 to 54 cycles at the same clock target. The final reciprocal estimate is read from the N_i register once the iteration count is reached (Fig. 4).

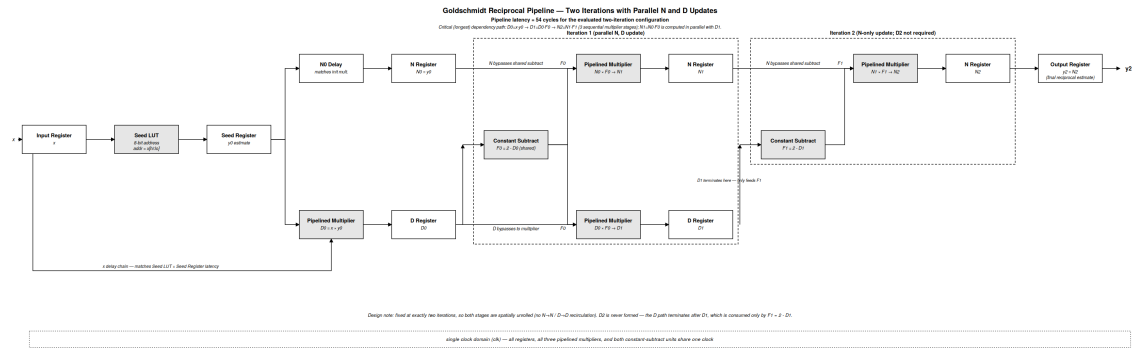


Fig. 4. Goldschmidt reciprocal architecture.

3.5 Hybrid LUT-Interpolation/Newton-Raphson

The Hybrid architecture (module name `reciprocal_hybrid9_1nr_q8_24` in the Q8.24 configuration) combines LUT-Interpolation with a single Newton-Raphson refinement stage. It first generates an interpolated reciprocal estimate using a 9-bit-address LUT-and-interpolation stage, providing a finer initial approximation than the 8-bit-address seed LUTs used by the standalone iterative architectures. The interpolated estimate is then refined using a single Newton-Raphson iteration of Eq. (3), rather than applying the iteration directly to a coarse LUT output. Since the interpolated

estimate has a smaller initial error, the refinement can achieve substantially higher accuracy while avoiding the deeper pipeline required by the standalone Newton-Raphson architecture. The complete hybrid design (Fig. 5) comprises four constituent modules: `reciprocal_interp9_q8_24` for the interpolation seed stage, `mult32x32_pipe_q8_24` for the pipelined multiplier (instantiated once per refinement multiplication), `nr_iterate_q8_24` for the single-iteration refinement, and `reciprocal_hybrid9_1nr_q8_24` as the top-level integration wrapper.

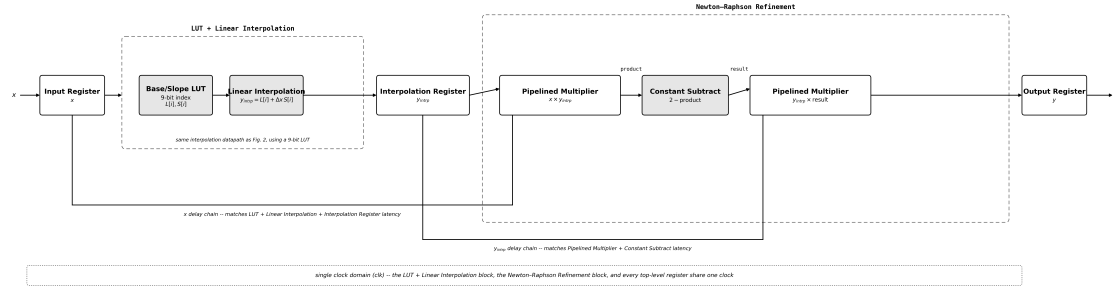


Fig. 5. Hybrid LUT-Interpolation/Newton-Raphson reciprocal architecture.

3.6 Common Hardware Configuration

All five architectures share a common set of implementation conventions to keep the comparison controlled. Each architecture is implemented as a fully pipelined datapath with registers inserted at every internal arithmetic stage boundary, so that after the pipeline fill latency, one new result is produced every clock cycle; this is what allows the Max Throughput (Mops/s) figures reported in Section 5 to equal the achieved $F_{\max}$ directly, rather than a fraction of it. All five architectures were synthesized and implemented for a Xilinx Artix-7 device (Nexys A7-100T development board [7], part xc7a100tcsg324-1) using the Xilinx Vivado design suite (2025.2.1), with out-of-context synthesis and full place-and-route implementation performed independently for each architecture/format combination at its own clock target (2.58 ns for the four 387.6 MHz-targeted architectures, 3.333 ns for the Hybrid architecture's 300 MHz target, per Table 1). The clock targets are therefore architecture-specific rather than identical: the Hybrid architecture was implemented against a less aggressive target than the other four, selected according to the timing requirements of its datapath (an interpolation stage followed by a Newton-Raphson refinement). Its latency and $F_{\max}$ figures should accordingly be read against that constraint rather than as results obtained under an identical clock requirement. The pipelined multipliers used throughout the iterative and hybrid datapaths map onto the device's DSP48E1 slices, each of which integrates a pre-adder, a 25×18 multiplier, and a configurable adder/accumulator with internal cascade paths [40]; the DSP utilization figures reported in Table 3 are counted in units of these slices. During implementation, multiple Vivado synthesis and implementation optimization strategies were used, including AggressiveExplore, ExtraTimingOpt, and Explore strategies, together with timing-oriented implementation directives, to obtain the best achievable timing under the stated architectural constraints. The same optimization methodology was applied across the architectures rather than relying on a single default implementation strategy. Two architecture/format combinations (Newton-Raphson and Goldschmidt) additionally required attribute-level intervention during physical synthesis (`dont_touch` constraints on

selected register stages) to prevent Vivado’s retiming optimizer from merging or removing intended pipeline register boundaries; this is discussed further in Section 4.2.

3.7 Algorithmic Complexity

Table 2 summarizes each architecture’s datapath in terms of primitive operation counts (multiplications, additions/subtractions, and memory reads) per complete reciprocal estimate. Direct LUT requires a single memory read and no arithmetic, while LUT-Interpolation adds one multiply and one add to the two table reads used for the stored breakpoint value and slope. Newton-Raphson performs two refinement iterations, with each iteration requiring two sequential multiplications and one subtraction, giving four multiplications and two subtractions per reciprocal estimate. Goldschmidt also performs two refinement iterations; its initialization requires one multiplication to form $D_0 = xy_0$, the first refinement computes $N_1 = N_0 F_0$ and $D_1 = D_0 F_0$ using two parallel multiplications, and the second refinement computes the final $N_2 = N_1 F_1$, giving four multiplications and two subtractions in total. The Hybrid architecture combines one interpolation stage with a single Newton-Raphson refinement, resulting in three multiplications and two additions/subtractions per reciprocal estimate. The parallel organization of Goldschmidt reduces sequential multiplier depth relative to Newton-Raphson without reducing the total arithmetic operation count.

Table 2. Algorithmic Operation Count per Reciprocal Estimate

Architecture	Mult.	Add/Sub	Mem. Reads	Seed Table Address
Direct LUT	0	0	1	12-bit address
LUT-Interpolation	1	1	2	11-bit address
Newton-Raphson	4	2	1	8-bit address
Goldschmidt	4	2	1	8-bit address
Hybrid	3	2	2	9-bit address

4 Experimental Methodology

4.1 Verification Methodology

Functional correctness and numerical accuracy for all five architectures, in both fixed-point formats, were verified against a common double-precision software reference model computing $1/x$ directly. Each architecture was exercised over one million verification vectors swept uniformly across the input domain $x \in [0.5, 2.5]$, with the bit-accurate RTL simulation output compared against the reference value at matching precision. This uniform sweep is a dense sample of the input domain rather than an enumeration of every representable input: Q16.16 has only 131,073 representable inputs in $[0.5, 2.5]$, so the sweep repeats values, and the reported maxima are maxima over the sampled vectors rather than proven worst-case bounds. Vectors are applied on consecutive clock cycles with the valid signal held high, so the sweep also exercises the one-cycle initiation interval, and each valid output is compared with the reference value of the input at the same position in the in-order stream. For each output sample, the signed error is defined in least-significant-bit (LSB) units as

$$e[n] = y_{hw}[n] - \text{round}\left(y_{ref}[n] 2^F\right) \quad (9)$$

Here, $y_{hw}[n]$ is the hardware/RTL output represented as a raw integer in the device’s Q-format, $y_{ref}[n]$ is the double-precision reference reciprocal, and F is the number of fractional bits for the format under test. For relative-error

calculation, the hardware output is converted to real-valued form as $y_{hw,real}[n] = y_{hw}[n] 2^{-F}$. The maximum relative error is then defined as

$$E_{max,rel} = \max_n \left(\frac{|y_{hw,real}[n] - y_{ref}[n]|}{|y_{ref}[n]|} \right) \times 100\% \quad (10)$$

Maximum relative error is used as the primary accuracy metric in the Pareto analysis of Section 6 because, unlike a raw LSB count, it is directly comparable across the two evaluated fixed-point formats: one LSB of Q8.24 (2^{-24}) and one LSB of Q16.16 (2^{-16}) represent different absolute quantities, so an LSB-based error metric cannot be used to rank accuracy across formats without first normalizing by format resolution, which the relative-error metric does implicitly. All pipelined architectures exhibit a warm-up transient during the first N_{cycles} output samples, corresponding to the pipeline fill period before the first valid result reaches the output register; these initial samples are excluded from all summary error statistics and are visualized explicitly in Section 5.3 (pipeline warm-up plots).

4.2 Timing Closure

Timing closure was the most implementation-intensive part of this work, particularly for the two full iterative architectures (Newton-Raphson and Goldschmidt), each of which required multiple revision cycles to close timing at their 387.6 MHz target on the Artix-7 fabric. Several recurring classes of timing violation were encountered and resolved across the iterative architectures' revision history. First, wide fixed-point adders synthesized as single-stage CARRY4 chains repeatedly failed to meet the target period and had to be manually split across pipeline stage boundaries (e.g., separating a cross-term adder from the final high-order sum in the pipelined multiplier). Second, Vivado's register-merge retiming optimization was found to silently undo manually inserted pipeline splits by merging what were intended as two distinct pipeline stages back into one combinational block. Register retiming is, in general, a valuable transformation that moves registers across combinational logic to balance path delays without changing observable circuit behavior [22], and is applied productively to FPGA pipelines in techniques such as C-slow retiming [39]; the difficulty encountered here was that automatic retiming, applied uniformly, merged register boundaries the design deliberately relied on as physical pipeline-stage markers. This was resolved by applying `dont_touch` (and, in one case, a synthesis-level `keep`) attributes to the registers marking the intended stage boundary, since a `shreg_extract="no"` attribute alone was found to be insufficient to prevent this class of retiming. Third, DSP48E1 cascade inference was found to occasionally fuse an intended register boundary directly into the DSP slice's internal C-port cascade path, bypassing the pipeline register the designer intended to be a hard stage boundary; this required explicit attribute-level or structural intervention to restore the intended pipeline stage count.

A related correctness (rather than timing) issue surfaced during this process: a one-cycle discrepancy in the reported latency of a signed dedicated-adder submodule (`dsp_add18x18_signed`) caused a systematic, low-probability functional error that was only caught by the full one-million-vector verification sweep described in Section 4.1 rather than by smaller directed test sets, underscoring the value of dense-sweep verification for pipelined fixed-point arithmetic where an off-by-one-cycle latency bug can silently corrupt only a small fraction of the output space. Once identified, the fix consisted of correcting the tracked `MULT_LATENCY` parameter used to align data with its corresponding valid/control signal through the pipeline, together with a corrected operand-port assignment inside the same submodule.

4.3 Power Estimation and Energy Derivation

Dynamic and static power values reported in Section 5.4 were obtained from Vivado's post-implementation power analysis for each architecture/format combination. These values are tool-based estimates rather than direct measurements

of board-level power consumption, and they do not rely on a validated switching-activity trace from the verification sweep. Because every evaluated architecture is fully pipelined to produce one valid output per clock cycle once its pipeline is filled, the sustained throughput of each architecture in operations per second is equal to its achieved $F_{\max}$. Energy per operation was subsequently derived from the reported total power and this throughput as

$$E_{op} = \frac{P_{total}}{F_{\max}} \quad (11)$$

This relationship treats the estimated total power as corresponding to operation at the achieved $F_{\max}$. It reproduces the reported Energy/op values in Table 6 for all ten architecture/format combinations to within rounding, and it is the basis of every Energy/op figure used in the Pareto analysis of Section 6. Reported latency in nanoseconds (Table 4) is computed separately from the achieved $F_{\max}$, as the product of the architecture's fixed pipeline depth in clock cycles and its target clock period:

$$L_{ns} = N_{cycles} \times T_{target} \quad (12)$$

using the target clock period from Table 1 rather than the achieved period, so that latency reflects the architecture's designed pipeline depth independent of any timing margin achieved beyond the target frequency. The achieved $F_{\max}$ is derived from the post-route slack at the target period as $F_{\max} = 1/(T_{target} - WNS)$. Latency in nanoseconds is therefore a design-point quantity at the target clock, whereas throughput is evaluated at $F_{\max}$, and the two are reported separately rather than combined into one timing figure.

5 Experimental Results

This section presents post-implementation results across resource utilization, timing and latency, numerical accuracy, and power/energy for all five architectures in both evaluated fixed-point formats, followed by a direct cross-format comparison in Section 5.5. A consolidated summary of all metrics is given in Section 6.5.

5.1 Resource Utilization

Table 3 and Fig. 6 report post-implementation resource utilization for all ten architecture/format combinations. The Direct LUT architecture consumes negligible slice LUT and flip-flop resources but requires **3.5 BRAMs in Q8.24, tied with LUT-Interpolation for the largest BRAM allocation among the evaluated configurations**, since it must store a full-precision reciprocal value at every one of its 4096 addressable entries. The two full iterative architectures (Newton-Raphson and Goldschmidt) show substantially higher slice LUT and flip-flop utilization than the table-based architectures, roughly 5 to 7 times that of LUT-Interpolation, reflecting their pipelined multiplier datapaths, and both consume the maximum 20 DSP48E1 slices allocated to the pipelined multiplier chains. The Hybrid architecture sits between the interpolation and full-iterative architectures on slice LUT/FF count, consistent with its interpolated seed followed by a single Newton-Raphson refinement iteration, while requiring roughly half the DSP count of the full iterative architectures (9 versus 20).

Table 3. Post-Implementation Resource Utilization

Architecture	Format	Slice LUT	Slice LUT%	FF	DSP	BRAM
Direct LUT	Q8.24	19	0.03%	84	0	3.5
Direct LUT	Q16.16	2	0.003%	51	0	2.5
LUT-Interpolation	Q8.24	159	0.25%	685	1	3.5
LUT-Interpolation	Q16.16	119	0.19%	647	1	2.0
Newton-Raphson	Q8.24	966	1.52%	3612	20	0.5
Newton-Raphson	Q16.16	849	1.34%	3340	20	0.5
Goldschmidt	Q8.24	846	1.33%	3656	20	0.5
Goldschmidt	Q16.16	722	1.14%	3385	20	0.5
Hybrid	Q8.24	768	1.21%	2626	9	1.0
Hybrid	Q16.16	799	1.26%	2586	9	1.0

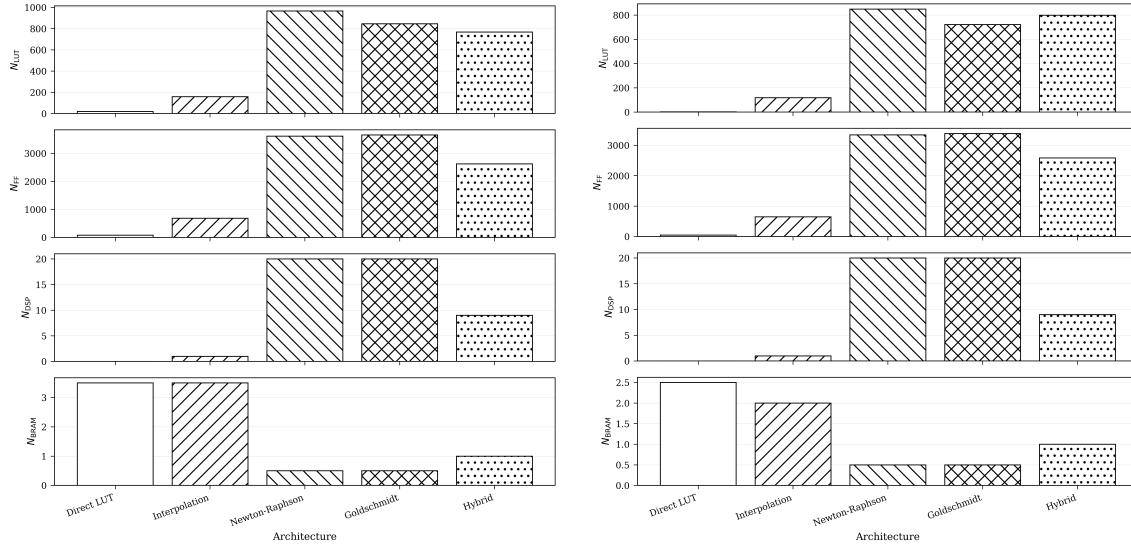


Fig. 6. Post-implementation slice LUT, FF, DSP, and BRAM utilization (resource counts) by architecture, Q8.24 (left) and Q16.16 (right)

5.2 Timing and Latency

Table 4 and Fig. 7 report the achieved maximum frequency ($F_{\max}$), worst negative slack (WNS) at the target clock period, pipeline depth, and corresponding latency calculated using Eq. (12). All ten architecture/format combinations achieve timing closure with positive slack at their respective target periods. The Direct LUT architecture has the shortest pipeline depth, requiring 5 cycles for Q8.24 and 3 cycles for Q16.16, resulting in the lowest absolute latency among the evaluated architectures. In contrast, the Newton-Raphson architecture requires the deepest pipeline at 69 cycles, reflecting the four sequentially dependent multiplication stages arising from its two refinement iterations, as described

in Section 3.3, Goldschmidt reduces the pipeline depth to 54 cycles by exploiting parallelism between intermediate multiplier operations, representing an approximately 22% reduction in pipeline depth relative to Newton-Raphson.

Table 4. Post-Implementation Timing and Latency

Architecture	Format	$F_{\max}$ (MHz)	WNS (ns)	Latency (cyc.)	Latency (ns)	Throughput (Mops/s)
Direct LUT	Q8.24	392.77	0.034	5	12.90	392.77
Direct LUT	Q16.16	519.21	0.654	3	7.74	519.21
LUT-Interpolation	Q8.24	429.74	0.253	16	41.28	429.74
LUT-Interpolation	Q16.16	424.45	0.224	16	41.28	424.45
Newton-Raphson	Q8.24	399.20	0.075	69	178.02	399.20
Newton-Raphson	Q16.16	392.62	0.033	69	178.02	392.62
Goldschmidt	Q8.24	396.35	0.057	54	139.32	396.35
Goldschmidt	Q16.16	394.94	0.048	54	139.32	394.94
Hybrid	Q8.24	316.06	0.169	40	133.32	316.06
Hybrid	Q16.16	305.06	0.055	40	133.32	305.06

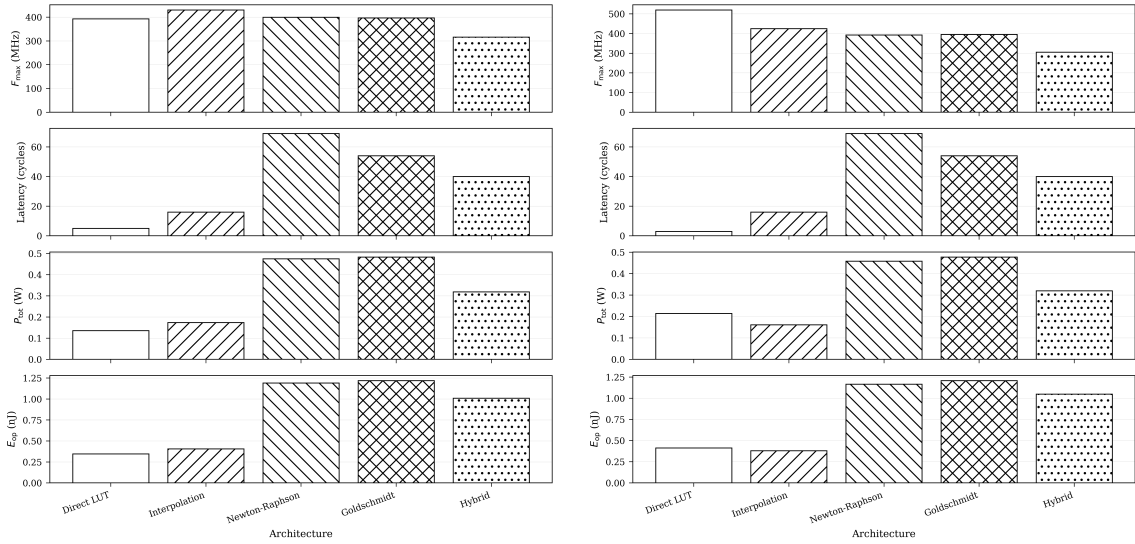


Fig. 7. Post-implementation $F_{\max}$ (MHz), latency (cycles), total power (W), and energy per operation (nJ/op) by architecture, Q8.24 (left) and Q16.16 (right)

5.3 Numerical Accuracy

Table ?? summarizes the four accuracy metrics defined in Section 4.1 for all ten architecture/format combinations. As expected from the architectural discussion in Sections 3.1 through 3.4, the two table-based architectures exhibit the largest maximum relative error (0.098% for Direct LUT and 0.0104% for LUT-Interpolation, both in Q8.24), while the

two full iterative architectures achieve error roughly 7×10^2 times smaller than interpolation and over 6×10^3 times smaller than the Direct LUT ($1.5 \times 10^{-5}\%$ for Newton-Raphson and $2.0 \times 10^{-5}\%$ for Goldschmidt, Q8.24). The Hybrid architecture, benefiting from a single refinement iteration applied to an already-interpolated seed, achieves a maximum relative error of $4.09 \times 10^{-4}\%$, roughly $25\times$ worse than the full iterative architectures but more than $25\times$ better than interpolation alone, consistent with the intermediate positioning described in Section 3.5.

Figs. 8-9 show the distribution of signed error across all five Q8.24 architectures, first in raw LSB units and then on a log-compressed $|e|$ scale to make the tails of the heavier-tailed distributions (Direct LUT, Hybrid) visible alongside the tightly concentrated iterative architectures. The corresponding Q16.16 distributions are shown in Figs. 14-15.

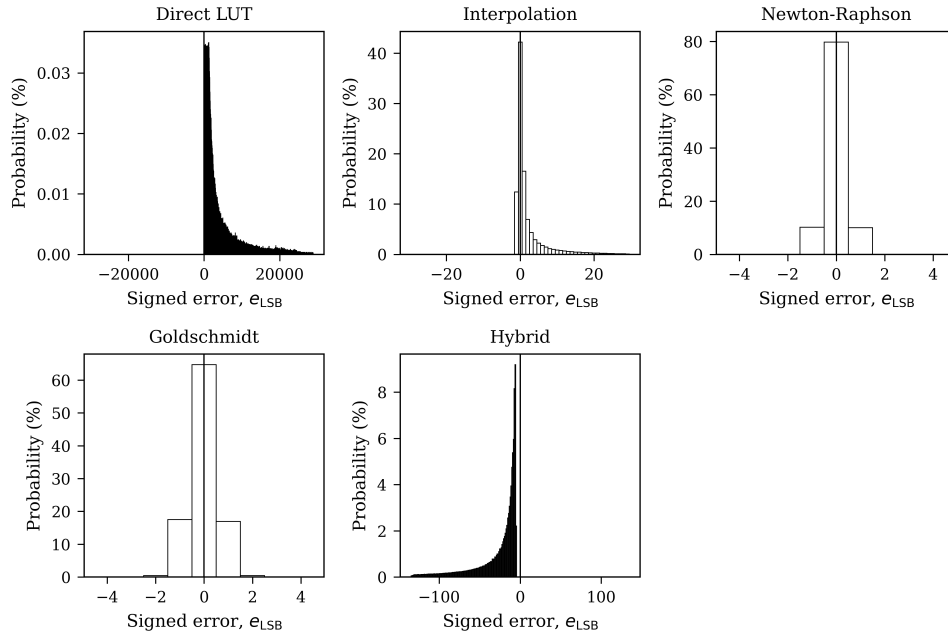


Fig. 8. Signed error distributions (error in LSB) by architecture, Q8.24.

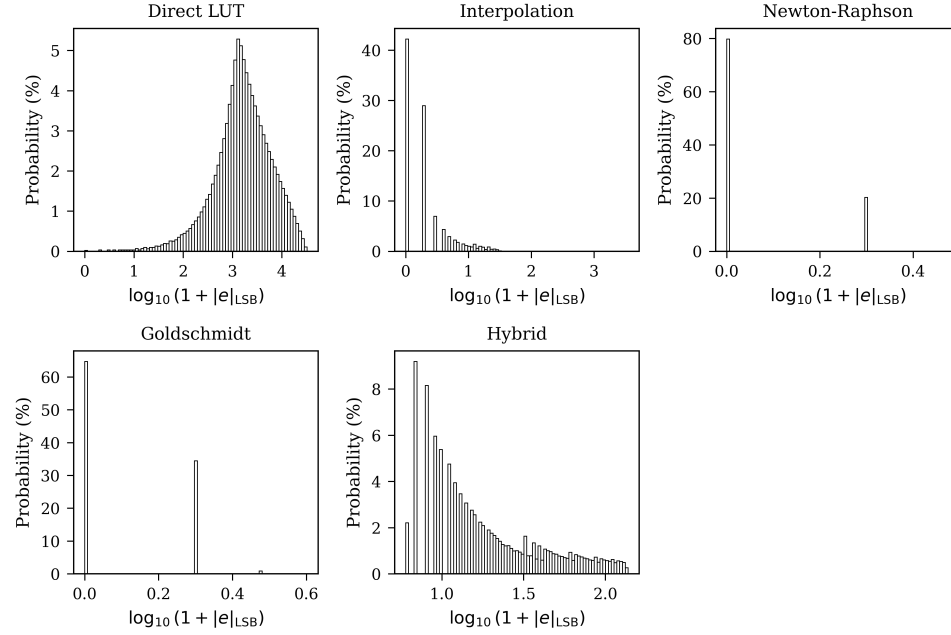


Fig. 9. Absolute error $|e|$ distributions by architecture, Q8.24 ($\log_{10}(1 + |e|)$ scale).

Fig. 10 decomposes the error population for each architecture into the fraction of outputs with zero LSB error relative to the quantized fixed-point reference and the distribution of nonzero errors. The zero-LSB fraction varies substantially across the architectures. Newton-Raphson and Goldschmidt produce exact agreement with the quantized fixed-point reference for a large fraction of test vectors, whereas the Hybrid architecture exhibits predominantly nonzero errors despite its substantially lower maximum relative error than the interpolation-only design.

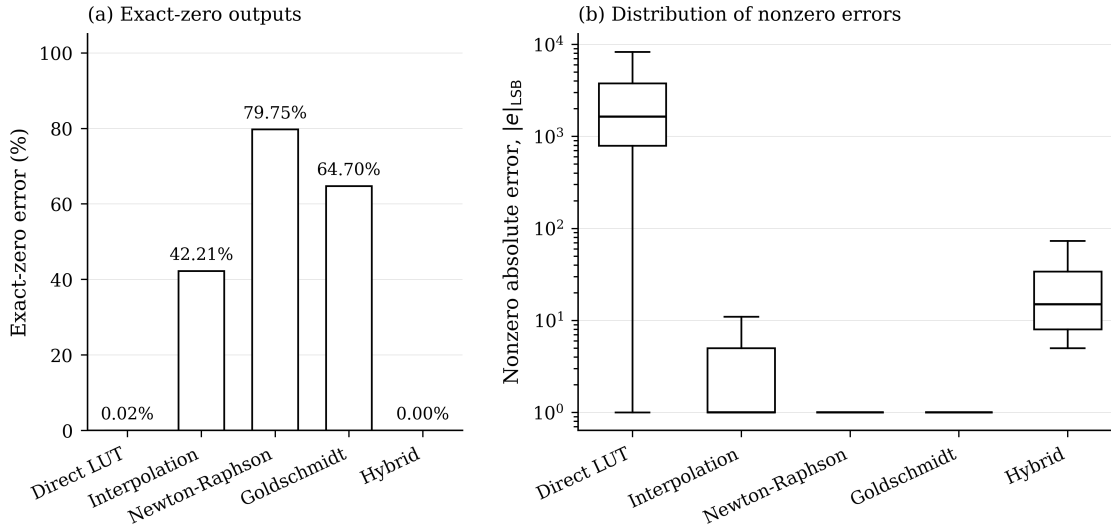


Fig. 10. Zero-LSB output fraction and distribution of nonzero signed errors (LSB) by architecture, Q8.24.

Fig. 11 shows how the maximum relative error varies as a function of the input value x across the domain $[0.5, 2.5]$. For all five architectures, error is concentrated toward the lower end of the input domain, where the local slope of $1/x$ is steepest and a fixed quantization step in x therefore maps to a larger quantization step in $1/x$; error decreases monotonically (Direct LUT) or in a sawtooth pattern tied to breakpoint/table boundaries (Interpolation, Goldschmidt, Hybrid) as x increases toward 2.5.

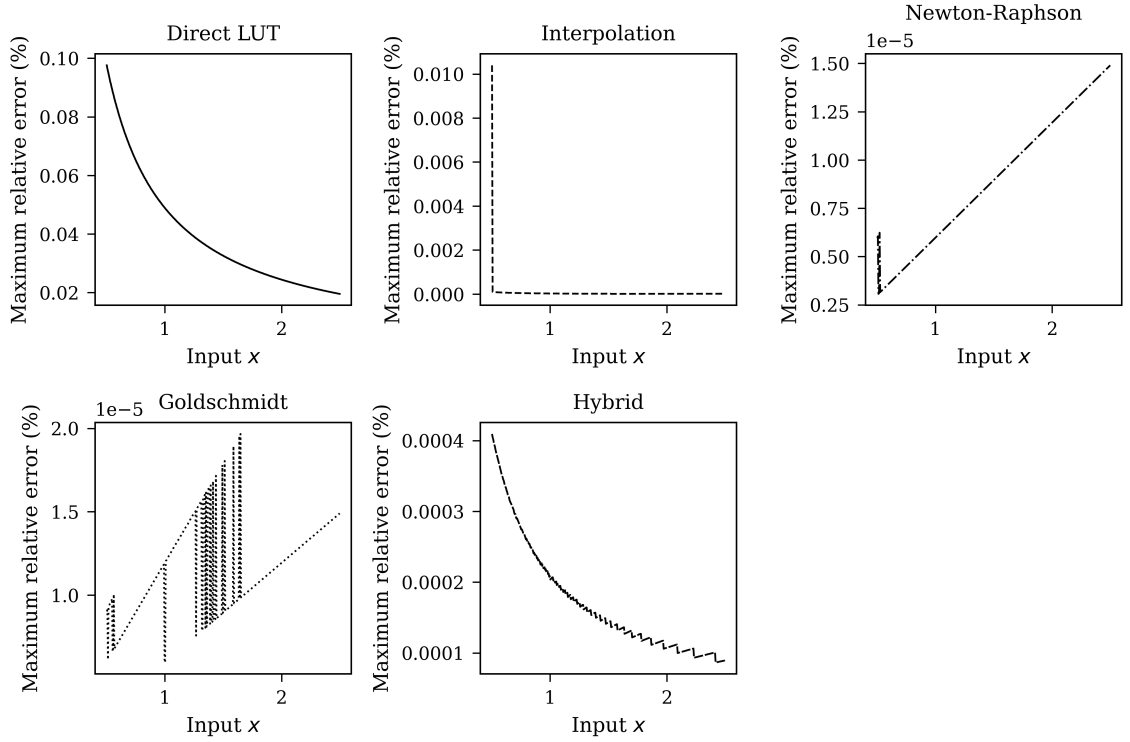


Fig. 11. Maximum relative error (%) as a function of input value x , by architecture, Q8.24.

Fig. 12 shows the pipeline warm-up transient discussed in Section 4.1: for each architecture, the first N_{cycles} output samples (shaded) correspond to the pipeline fill period and do not represent valid steady-state outputs; these samples are excluded from all summary statistics reported in Table ?? Fig. 13 presents the complementary cumulative view, showing absolute error as a function of output percentile across the full one-million-vector sweep, which makes clear that all five architectures' worst-case error is concentrated in a small percentile tail rather than being representative of typical-case behavior.

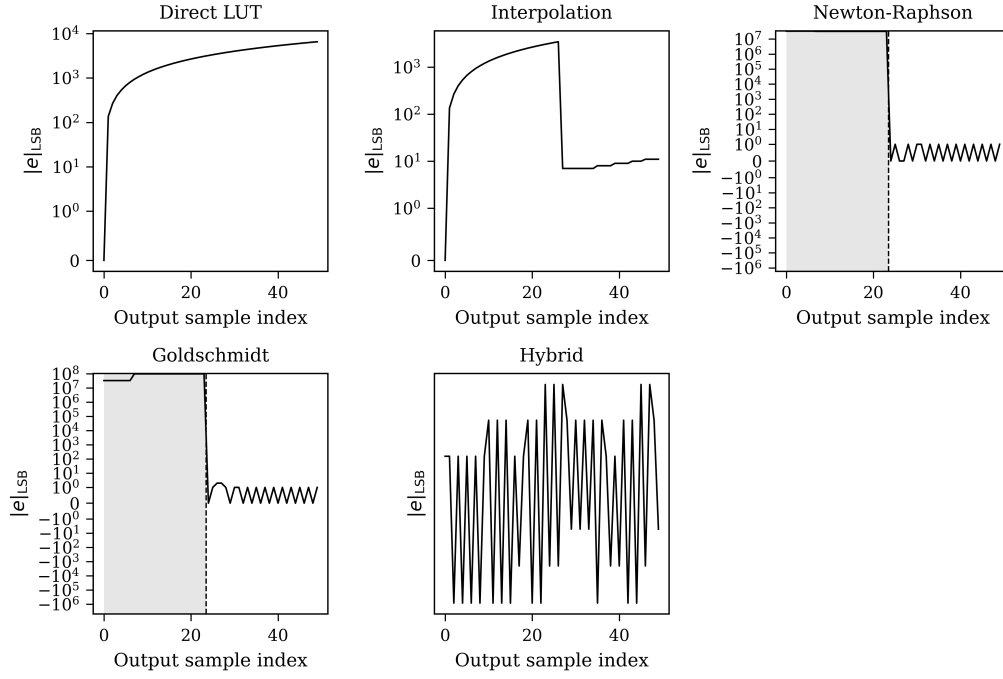


Fig. 12. Output error during the pipeline warm-up transient by architecture (shaded region: pipeline fill period), Q8.24.

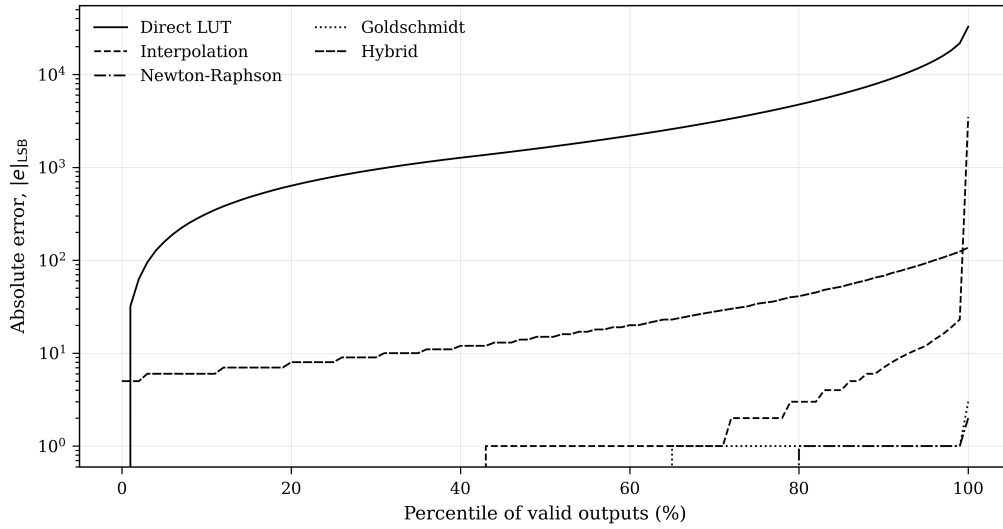


Fig. 13. Absolute error (real-valued) as a function of output percentile, all architectures, Q8.24.

The corresponding Q16.16 error-distribution, worst-case, warm-up, and percentile figures are reproduced in Figs. 14-19 for direct comparison against the Q8.24 results above; the cross-format differences visible between these figure pairs are discussed quantitatively in Section 5.5.

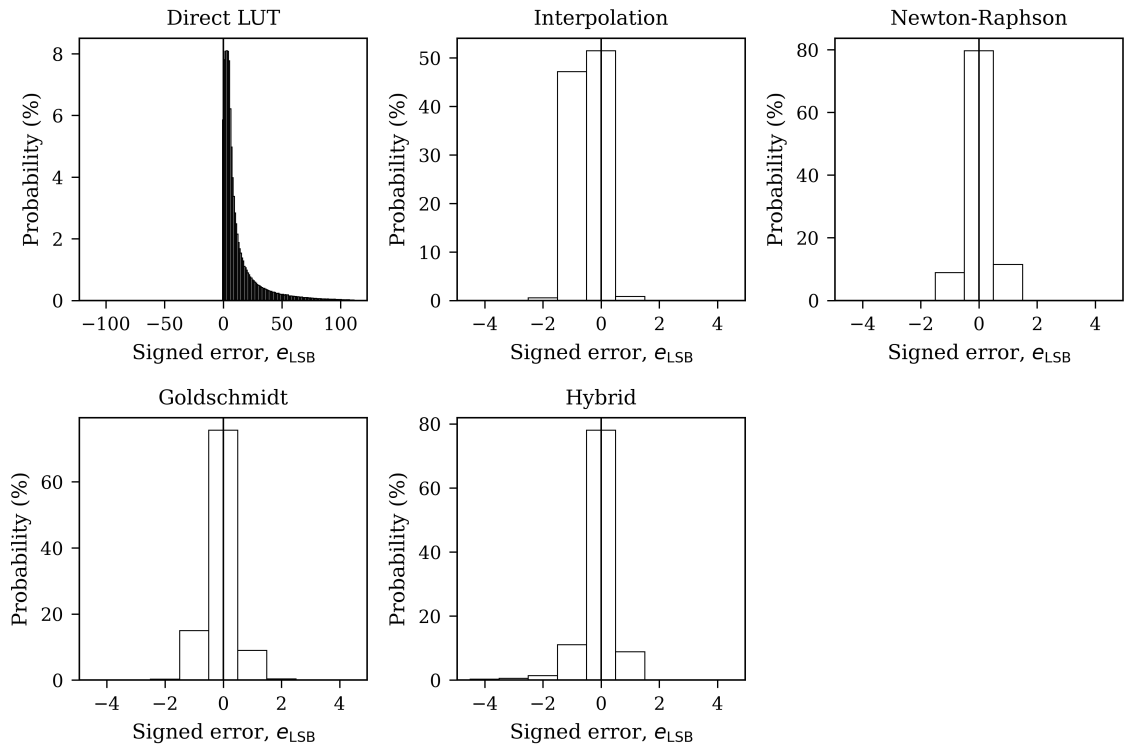


Fig. 14. Signed error distributions (error in LSB) by architecture, Q16.16.

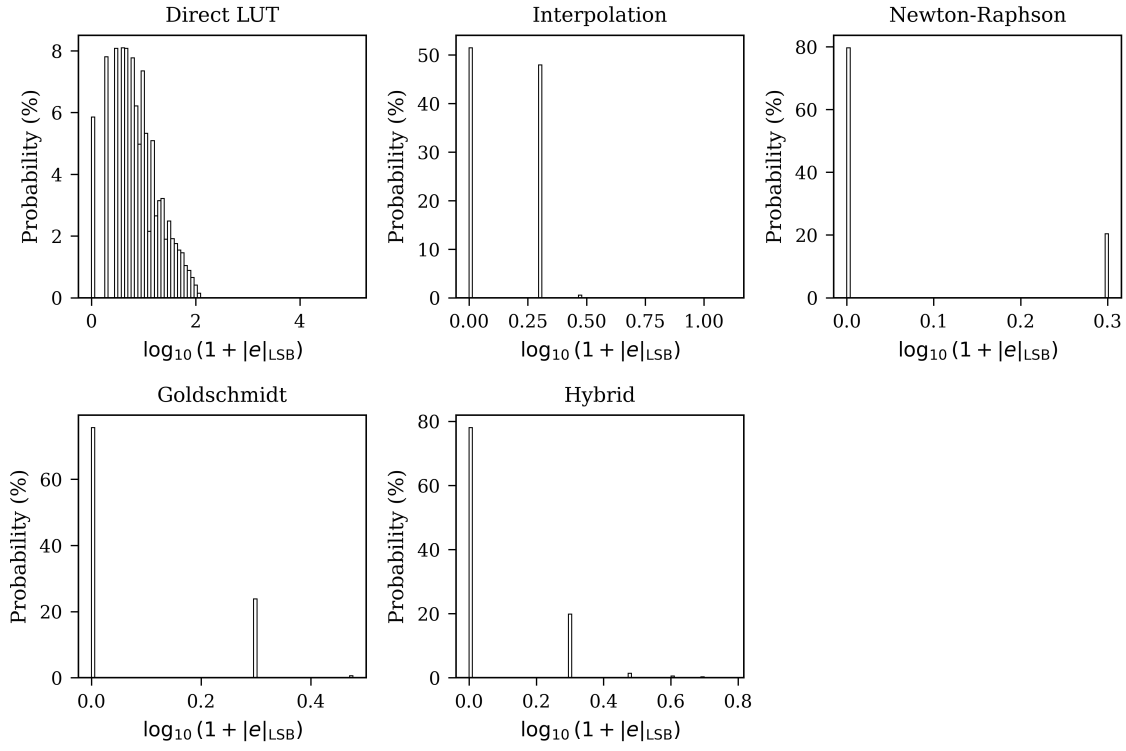


Fig. 15. Absolute error $|e|$ distributions by architecture, Q16.16 ($\log_{10}(1 + |e|)$ scale).

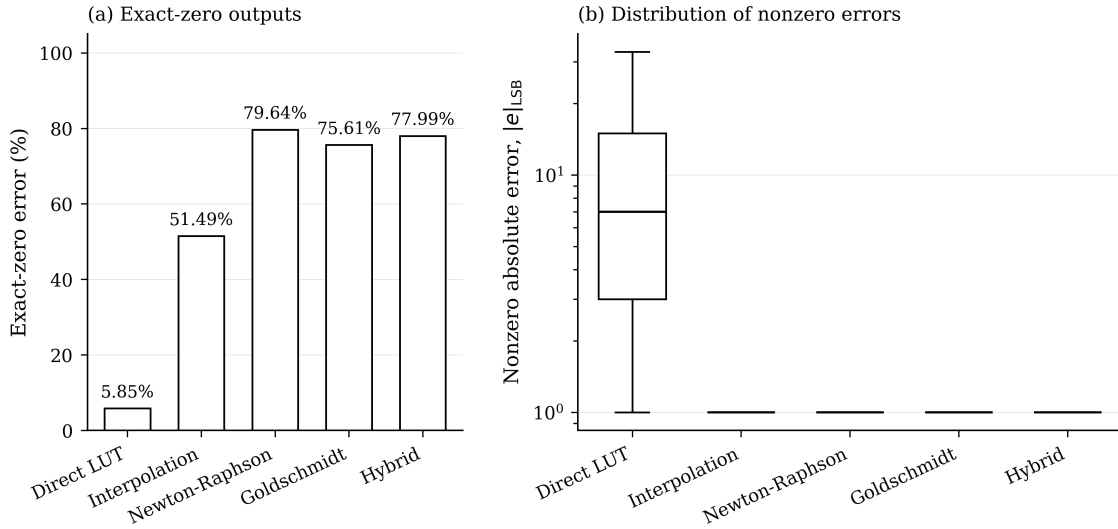


Fig. 16. Zero-LSB output fraction and distribution of nonzero signed errors (LSB) by architecture, Q16.16.

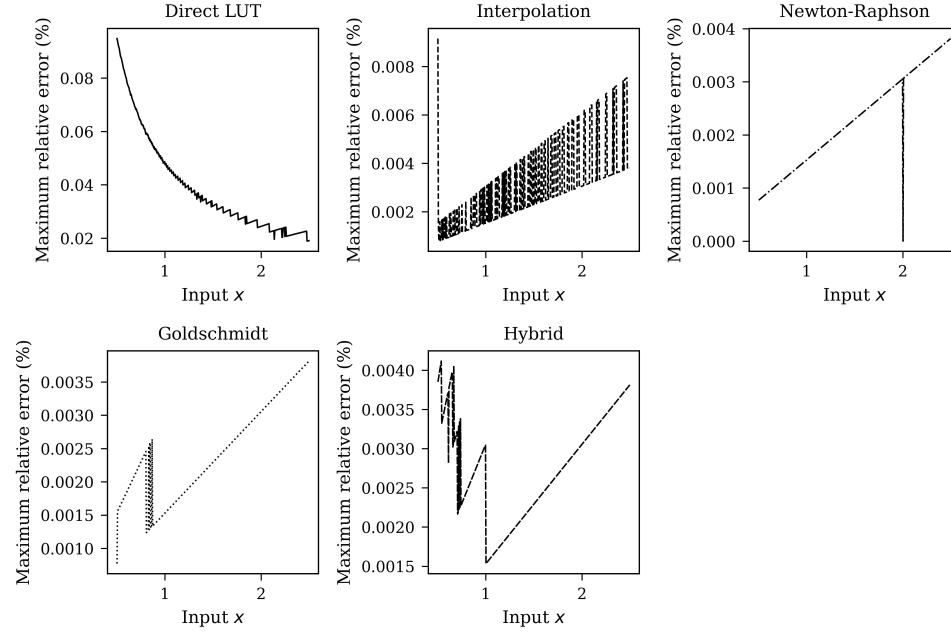


Fig. 17. Maximum relative error (%) as a function of input value x , by architecture, Q16.16.

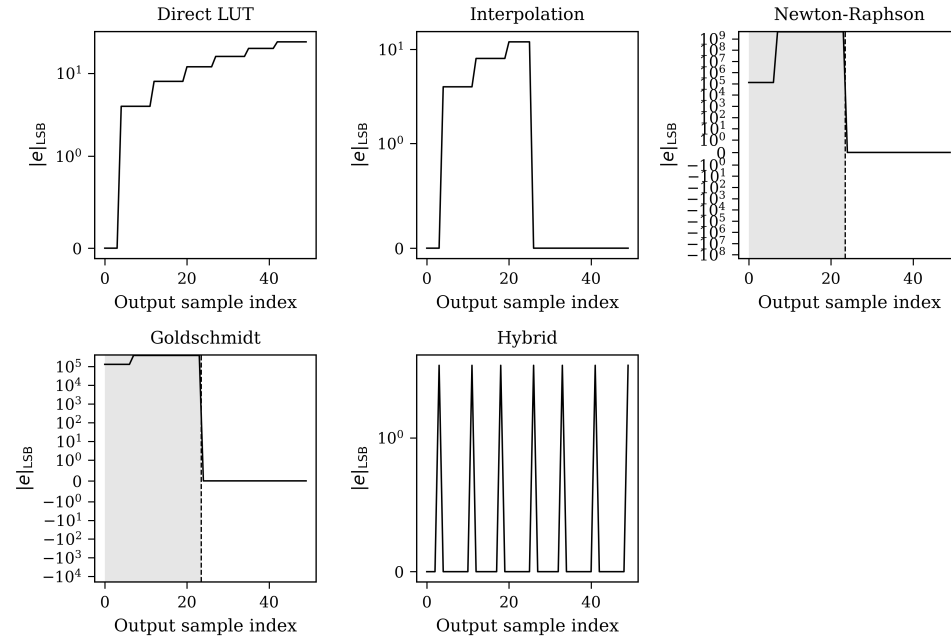


Fig. 18. Output error during the pipeline warm-up transient by architecture (shaded region: pipeline fill period), Q16.16.

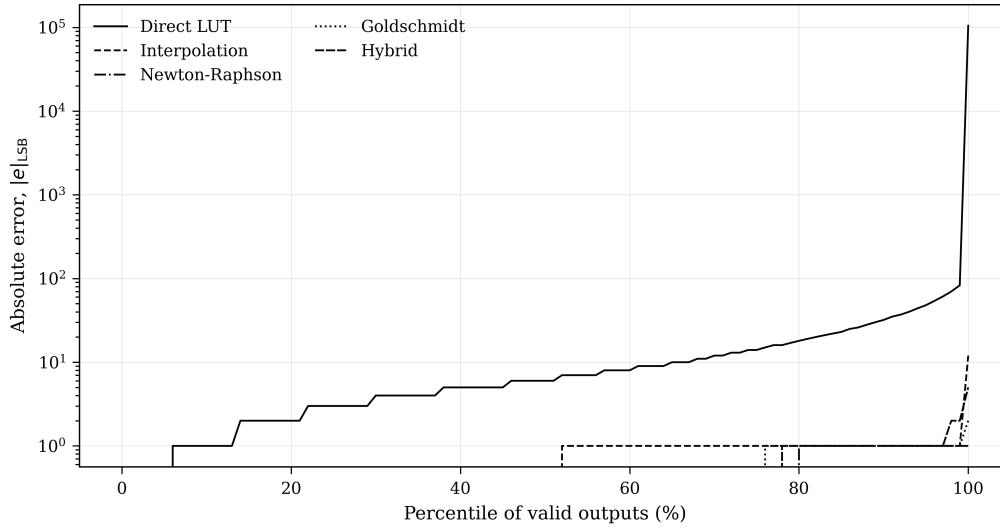


Fig. 19. Absolute error (real-valued) as a function of output percentile, all architectures, Q16.16.

5.4 Power and Energy

Table 6 and Fig. 7 report dynamic, static, and total power estimates together with the derived energy per operation (Eq. (11)) for all ten architecture/format combinations. The highest total power estimates (0.458 to 0.483 W) are observed for the two multiplier-intensive iterative architectures, which also exhibit the highest DSP and flip-flop utilization; since energy per operation is the total power divided by the valid-output rate (Eq. (11)), they also have the highest energy per operation (1.167 to 1.219 nJ/op). In Q8.24 the Direct LUT architecture has the lowest energy per operation (0.346 nJ/op), reflecting its minimal dynamic power estimate; in Q16.16 the lowest value belongs to LUT-Interpolation (0.379 nJ/op, against 0.412 nJ/op for Direct LUT). Static power estimates vary only slightly (0.091 to 0.092 W) across the ten configurations.

Table 6. Post-Implementation Power Estimates and Derived Energy

Architecture	Format	Dynamic P (W)	Static P (W)	Total P (W)	Energy/op (nJ)
Direct LUT	Q8.24	0.045	0.091	0.136	0.346
Direct LUT	Q16.16	0.122	0.092	0.214	0.412
LUT-Interpolation	Q8.24	0.083	0.091	0.174	0.405
LUT-Interpolation	Q16.16	0.070	0.091	0.161	0.379
Newton-Raphson	Q8.24	0.383	0.092	0.475	1.190
Newton-Raphson	Q16.16	0.366	0.092	0.458	1.167
Goldschmidt	Q8.24	0.391	0.092	0.483	1.219
Goldschmidt	Q16.16	0.385	0.092	0.477	1.208
Hybrid	Q8.24	0.227	0.092	0.319	1.009
Hybrid	Q16.16	0.228	0.092	0.320	1.049

5.5 Cross-Format Comparison

Figs. 20-23 directly compare Q8.24 and Q16.16 across achieved $F_{\max}$ and latency, slice LUT/DSP resource usage, total power and energy per operation, and full four-metric resource utilization, respectively. Two cross-format effects are evident. First, Direct LUT achieves markedly higher $F_{\max}$ in Q16.16 than in Q8.24, increasing from 392.8 MHz to 519.2 MHz, whereas LUT-Interpolation is essentially unchanged (429.7 MHz versus 424.5 MHz). This difference is consistent with the narrower output representation and the resulting implementation of the BRAM/output datapath, although the exact timing improvement depends on the post-implementation critical paths. Newton-Raphson, Goldschmidt, and Hybrid likewise show comparatively small $F_{\max}$ differences between the two formats, reflecting the greater influence of their pipelined arithmetic datapaths on the achieved timing.

Second, the iterative and hybrid architectures exhibit substantially different accuracy behavior across the two formats. Newton-Raphson and Goldschmidt, both seeded from 8-bit-address coarse lookup tables, show substantially higher maximum relative error in Q16.16 than in Q8.24, by approximately two orders of magnitude. The Hybrid architecture, which uses a 9-bit-address interpolated seed followed by a Newton-Raphson refinement, also shows a higher maximum relative error in Q16.16. This behavior is consistent with the reduced fractional precision of Q16.16, which provides fewer bits for representing intermediate and final correction terms throughout the fixed-point refinement process.

These results illustrate that the effect of fixed-point format extends beyond numerical representation alone and influences the measured timing and accuracy characteristics of the architectures. Consequently, the relative trade-offs among accuracy, latency, and energy depend on both the reciprocal algorithm and the fixed-point format used by the surrounding system.

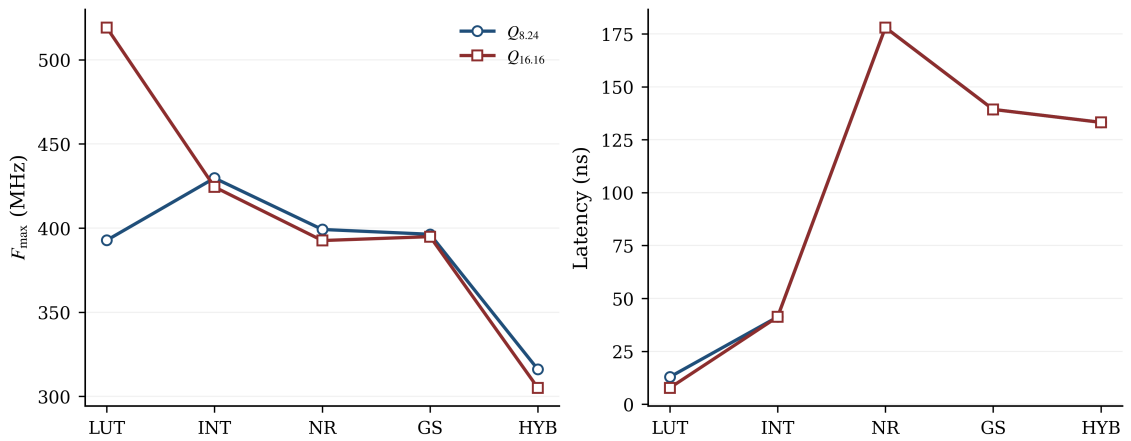


Fig. 20. Achieved $F_{\max}$ (MHz) and latency (ns) by architecture, Q8.24 versus Q16.16.

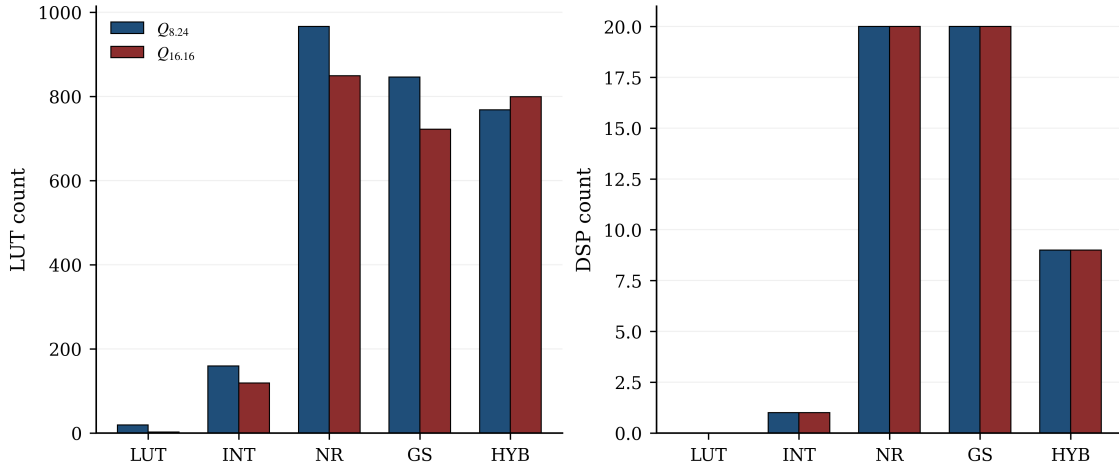


Fig. 21. Slice LUT and DSP utilization (resource counts) by architecture, Q8.24 versus Q16.16.

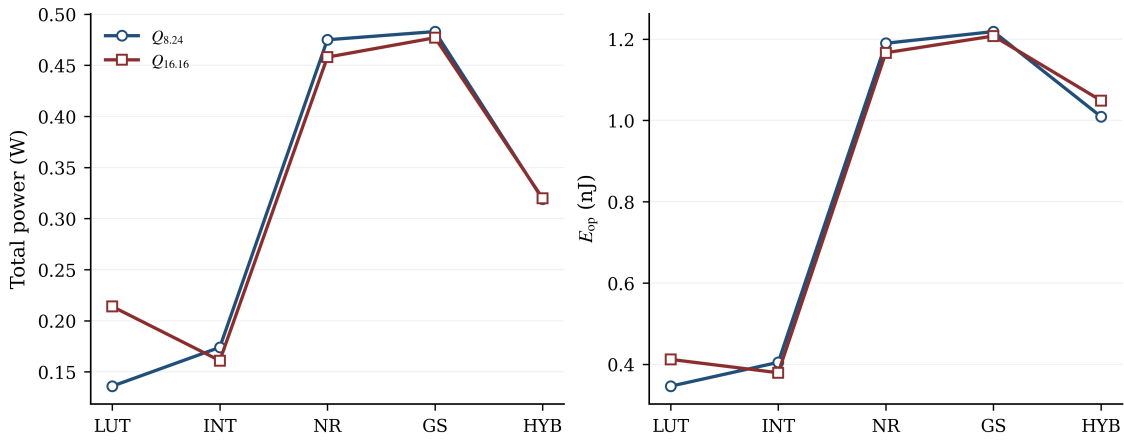


Fig. 22. Total power (W) and energy per operation (nJ/op) by architecture, Q8.24 versus Q16.16.

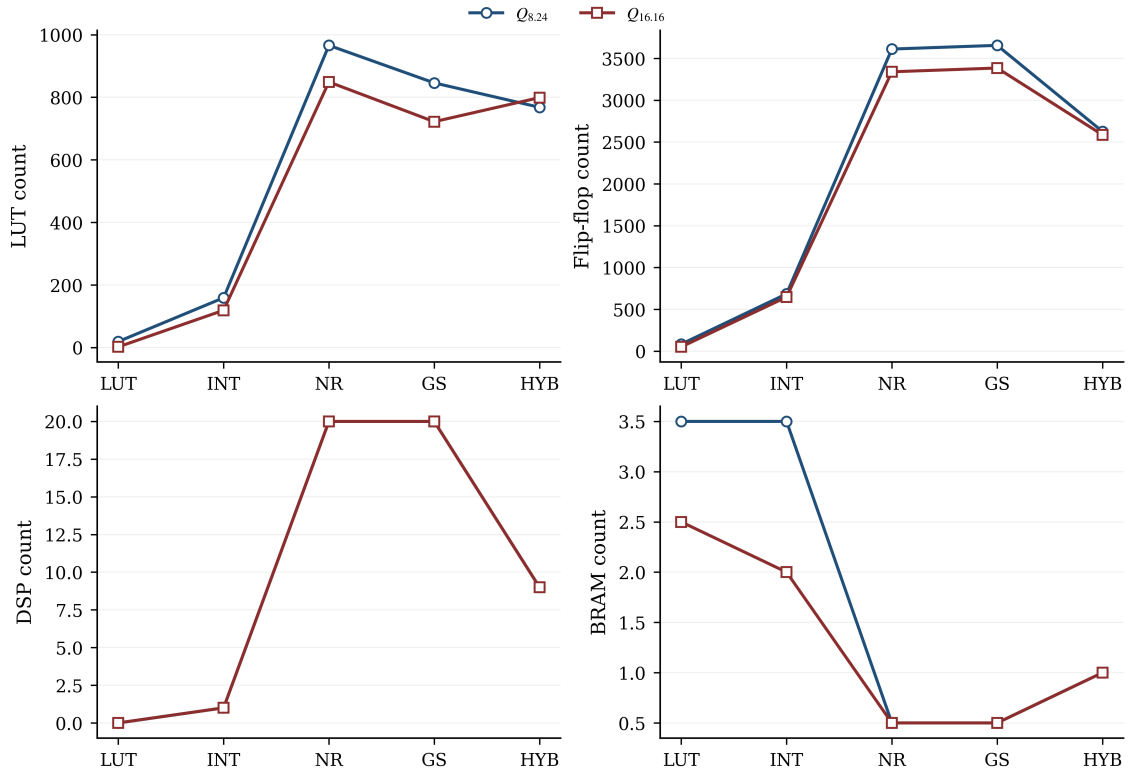


Fig. 23. Full resource utilization (slice LUT, FF, DSP, BRAM counts) by architecture, Q8.24 versus Q16.16.

6 Pareto Trade-off Analysis

6.1 Pareto Formulation

The results of Section 5 show that no single architecture dominates the others across every metric of interest. The relative ordering depends on the fixed-point format and the metric considered: the iterative architectures provide substantially lower numerical error than the table-based approaches, but generally incur higher latency and energy per operation, whereas the Direct LUT architecture provides the shortest latency with low energy per operation at the cost of higher approximation error. To characterize these trade-offs rigorously rather than anecdotally, this section applies a three-objective Pareto analysis over maximum relative error, latency, and energy per operation:

$$\text{minimize } \{E_{\max,rel}, L_{ns}, E_{op}\} \quad (13)$$

Hardware resource utilization (slice LUT, FF, DSP, BRAM) is deliberately reported separately (Table 3) rather than folded into the Pareto objective set above. A composite resource metric would require assigning relative weights across four resource types with different device-level scarcity and cost characteristics (e.g., BRAM versus DSP), and any such weighting would introduce a design choice into what is intended to be an objective, data-driven comparison; a design's resource cost is instead read directly from Table 3 alongside its position on the Pareto front. Given this three-objective formulation, an architecture A is said to dominate an architecture B if A is no worse than B on all three objectives and

strictly better on at least one:

$$A \succ B \iff (\forall k \in \{E_{rel}, L, E_{op}\} : A_k \leq B_k) \wedge (\exists k : A_k < B_k) \quad (14)$$

An architecture is Pareto-optimal (non-dominated) if no other evaluated architecture dominates it under this relation. Energy-delay product ($EDP = E_{op} \times L_{ns}$) is computed separately as a secondary, derived metric (Section 6.4) rather than substituted for latency and energy as a combined objective, since two architectures with identical EDP can have substantially different individual energy and latency characteristics that matter for system-level integration (e.g., a hard real-time latency budget versus a battery energy budget), and collapsing the two into one number would discard that distinction.

6.2 Pareto Fronts

Fig. 24 plots each architecture's position in the latency-energy plane for both fixed-point formats, with marker size inversely related to maximum relative error: larger markers indicate lower maximum relative error. A single global marker-size scale is shared across both panels so that marker sizes are directly comparable between the Q8.24 and Q16.16 results rather than only within each format. Applying the dominance relation of Eq. (14) strictly to all five architectures within each format shows that all five architectures are non-dominated among the evaluated configurations in both Q8.24 and Q16.16: no evaluated architecture is simultaneously no worse than another architecture on all three objectives while being strictly better on at least one. Thus, each architecture represents a distinct trade-off point in the three-dimensional accuracy-latency-energy objective space, and the resulting Pareto set reflects different combinations of numerical accuracy, latency, and energy per operation rather than a single architecture dominating across all three objectives. The pairwise comparisons are listed in Appendix ???. This non-dominance holds only among the evaluated configurations; because the seed-table size, iteration count, and interpolation resolution are fixed, it does not establish optimality over the wider architecture design space.

This strict result should be read with one qualification in Q16.16. The maximum relative errors of Newton-Raphson, Goldschmidt, and Hybrid in that format (0.003815%, 0.003814%, and 0.004118%) differ by at most about 8% and lie at the scale of a single output LSB (Appendix ??). The non-dominance of the Newton-Raphson/Hybrid and Goldschmidt/Hybrid pairs therefore rests on these small error differences, since Hybrid has lower latency and lower energy per operation than both. If errors within a tolerance of one output LSB are treated as equivalent (one LSB of Q16.16, 2^{-16} , corresponds to a relative error of $7.6 \times 10^{-4}\%$ at the worst-case point $x = 0.5$), Hybrid would dominate both iterative architectures in Q16.16, and the non-dominated set in that format would reduce to Direct LUT, LUT-Interpolation, and Hybrid. In Q8.24 the corresponding error differences exceed an order of magnitude, and the five-architecture Pareto set is unaffected by such a tolerance.

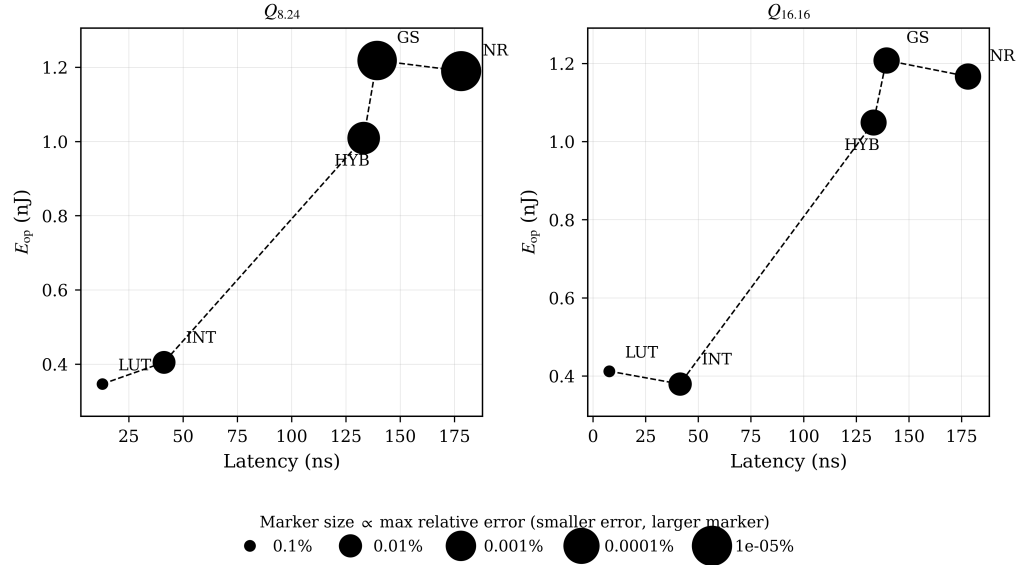


Fig. 24. Pareto trade-off in the latency–energy plane (latency in ns, energy per operation in nJ/op) by architecture, $Q_{8.24}$ (left) and $Q_{16.16}$ (right). Marker size is inversely related to maximum relative error (%) on a shared global scale across both panels; all five architectures are mutually non-dominated in both formats under strict dominance.

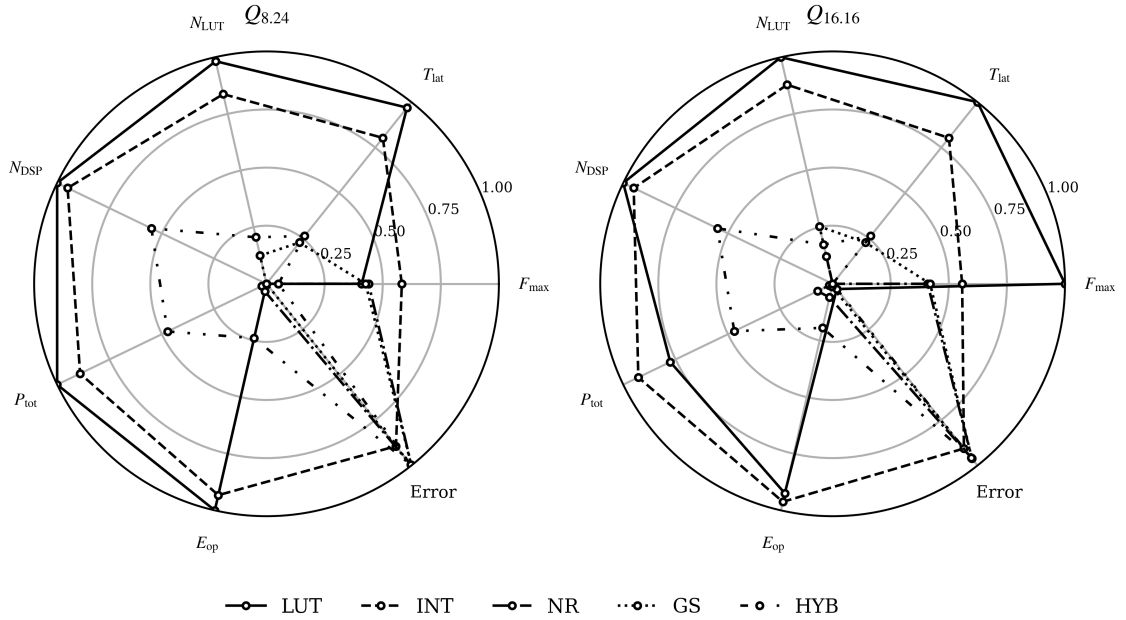


Fig. 25. Pareto score radar plot for the evaluated architectures.

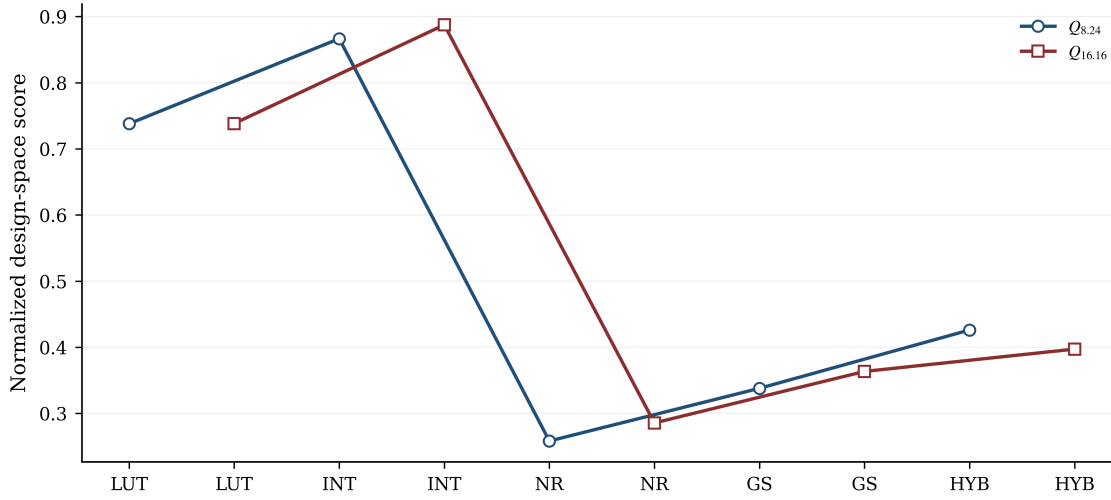


Fig. 26. Pareto score line plot for the evaluated architectures.

6.3 Trade-off Analysis

Within the Q8.24 non-dominated set, Direct LUT occupies the low-latency and low-energy region of the trade-off surface, with a latency of 12.9 ns, energy of 0.346 nJ/op, and maximum relative error of 0.098%. Newton-Raphson occupies the high-accuracy region, achieving $1.5 \times 10^{-5}\%$ maximum relative error at 178.0 ns latency and 1.190 nJ/op. Goldschmidt provides comparable numerical accuracy, with a maximum relative error of $2.0 \times 10^{-5}\%$, while reducing latency by approximately 22% to 139.3 ns. This latency reduction is accompanied by a small increase in energy per operation, from 1.190 to 1.219 nJ/op (approximately 2.4%). Thus, the measured Goldschmidt implementation represents a lower-latency alternative to Newton-Raphson when latency is prioritized over a small difference in energy per operation.

The Hybrid architecture occupies an intermediate region of the trade-off surface. It achieves 133.32 ns latency and 1.009 nJ/op, both lower than the corresponding values of the full iterative architectures, while achieving a maximum relative error of $4.09 \times 10^{-4}\%$. This represents more than an order-of-magnitude improvement over interpolation-only accuracy ($1.04 \times 10^{-2}\%$). The result demonstrates the distinct trade-off obtained by combining a 9-bit-address interpolated seed with a single Newton-Raphson refinement, providing an intermediate accuracy, latency, energy, and DSP-utilization point between table-based and fully iterative approaches. This intermediate accuracy positioning is specific to Q8.24; in Q16.16 the hybrid error is within about 8% of the iterative architectures, as discussed above.

6.4 Energy-Delay Product

Table 7 reports energy-delay product ($EDP = E_{op} \times L_{ns}$) for all ten architecture/format combinations as a secondary combined figure of merit illustrating the interaction between energy and latency, per the reasoning in Section 6.1. EDP reproduces the same broad ranking as the individual latency and energy metrics, with Direct LUT lowest and the full iterative architectures highest. It is a derived metric rather than a universal physical objective, and the three-objective Pareto analysis above remains the primary basis for comparison.

Table 7. Energy-Delay Product (EDP = Energy/op $\times$ Latency)

Architecture	Format	Energy/op (nJ)	EDP (nJ·ns)
Direct LUT	Q8.24	0.346	4.46
Direct LUT	Q16.16	0.412	3.19
LUT-Interpolation	Q8.24	0.405	16.72
LUT-Interpolation	Q16.16	0.379	15.66
Newton-Raphson	Q8.24	1.190	211.80
Newton-Raphson	Q16.16	1.167	207.68
Goldschmidt	Q8.24	1.219	169.79
Goldschmidt	Q16.16	1.208	168.24
Hybrid	Q8.24	1.009	134.56
Hybrid	Q16.16	1.049	139.85

6.5 Consolidated Summary of Results

Table 8 consolidates the key metrics from Tables 3-6 and Table 7 into a single reference table, for readers who want the complete set of headline figures for all ten architecture/format combinations without cross-referencing multiple tables.

Table 8. Consolidated Summary of All Reported Metrics, All Architectures and Formats

Architecture	Format	Slice LUT%	DSP	Latency (ns)	Max Rel. Err (%)	Total P (W)	Energy/op (nJ)	EDP (nJ·ns)
Direct LUT	Q8.24	0.03%	0	12.90	0.097596	0.136	0.346	4.46
Direct LUT	Q16.16	0.003%	0	7.74	0.094787	0.214	0.412	3.19
LUT-Interpolation	Q8.24	0.25%	1	41.28	0.010396	0.174	0.405	16.72
LUT-Interpolation	Q16.16	0.19%	1	41.28	0.009156	0.161	0.379	15.66
Newton-Raphson	Q8.24	1.52%	20	178.02	0.000015	0.475	1.190	211.80
Newton-Raphson	Q16.16	1.34%	20	178.02	0.003815	0.458	1.167	207.68
Goldschmidt	Q8.24	1.33%	20	139.32	0.000020	0.483	1.219	169.79
Goldschmidt	Q16.16	1.14%	20	139.32	0.003814	0.477	1.208	168.24
Hybrid	Q8.24	1.21%	9	133.32	0.000409	0.319	1.009	134.56
Hybrid	Q16.16	1.26%	9	133.32	0.004118	0.320	1.049	139.85

7 Discussion

7.1 Architecture-Level Trade-offs

The results of Sections 5 and 6 illustrate several distinct architecture-level trade-offs. For applications in which latency and energy are important constraints and an error on the order of **0.1%** is acceptable, the **Direct LUT** architecture provides the shortest pipeline and negligible logic utilization, while requiring one of the largest BRAM footprints among the evaluated designs. For the **Q8.24** results, applications requiring maximum relative error below approximately $10^{-4}\%$ are served by the full iterative architectures, which provide substantially higher numerical accuracy than the table-based approaches. Within this accuracy range, the latency advantage of **Goldschmidt** over Newton-Raphson (Section 6) is

accompanied by an energy-per-operation increase of approximately **2.4% in Q8.24 and 3.5% in Q16.16**. The **hybrid** architecture provides a distinct intermediate trade-off by combining a 9-bit-address interpolated seed with a single Newton-Raphson refinement. In Q8.24 it achieves substantially lower error than interpolation alone while reducing the hardware cost of full iterative refinement, with approximately **75% of the Newton-Raphson latency and 45% of the DSP utilization of either full iterative architecture**. In Q16.16, where the iterative and hybrid errors are within about 8% of one another, the Hybrid architecture offers similar accuracy with lower latency, energy, and DSP usage.

7.2 Applicability

The reciprocal approximation architectures evaluated in this work are applicable wherever a fixed-point, division-free reciprocal is required as a building block within a larger FPGA arithmetic pipeline. Representative applications include normalization stages in adaptive and statistical signal processing, such as computing a z-score or an Ornstein-Uhlenbeck mean-reversion signal, where division by a continuously updated scale or standard-deviation term may be required; coordinate and vector normalization in graphics and radar/sonar signal chains, where a reciprocal can precede a subsequent multiplication; and floating-point division and mantissa-normalization units, where the reciprocal is applied to a range-reduced significand. In these settings, the appropriate architecture depends on the required combination of **numerical precision, latency, throughput, resource utilization, and energy**, with the evaluated designs occupying different regions of the corresponding trade-off space rather than a single architecture being universally optimal.

7.3 Limitations

Several limitations should be considered when interpreting and generalizing the results. First, all experiments use a single input domain, $x \in [0.5, 2.5]$. Although this range represents the bounded reciprocal domain considered in Section 2.1 and captures the characteristic error concentration near the lower bound discussed in Section 5.3, the evaluation does not include alternative or non-uniform input distributions. Such distributions may alter the observed accuracy and performance trade-offs among the architectures.

Second, the evaluation is limited to a single FPGA family, Xilinx Artix-7. The relative costs of BRAM, DSP, slice LUT, and FF resources are device-dependent; consequently, the specific resource-utilization trade-offs reported in Table 3 may differ on other FPGA families or vendor platforms. The timing and power characteristics may likewise change with the underlying device architecture and implementation resources. In addition, the reported power values are post-implementation tool estimates rather than board-level measurements, and the Hybrid architecture was evaluated against a less aggressive clock target than the other four architectures.

Third, the study does not include a direct comparison with vendor-supplied divider or reciprocal IP cores, such as Xilinx LogiCORE-class implementations, nor with CORDIC-based reciprocal/division computation [2, 37], which trades the multiplier-centric datapaths evaluated here for a shift-and-add rotation structure well suited to FPGA fabrics with limited DSP resources. Such comparisons would provide additional baselines for evaluating the custom architectures against off-the-shelf and multiplier-free solutions and are therefore left for future work.

Finally, the evaluated iterative architectures use fixed iteration configurations: **two Newton-Raphson iterations, two Goldschmidt iterations, and one Newton-Raphson refinement iteration following LUT interpolation in the Hybrid architecture**. The seed-table sizes and interpolation resolutions are likewise fixed for the evaluated configurations. A broader exploration varying iteration count, seed-table size, and interpolation resolution could expose additional points in the accuracy, latency, resource, and energy trade-off space beyond the five architectures evaluated in this study.

8 Conclusion

This work presented a post-implementation comparative evaluation of five fixed-point reciprocal approximation architectures (Direct LUT, LUT with Linear Interpolation, Newton-Raphson, Goldschmidt, and a hybrid LUT-Interpolation/Newton-Raphson architecture) implemented in Q8.24 and Q16.16 fixed-point formats on a Xilinx Artix-7 XC7A100T FPGA. All architectures were evaluated using a common input domain of $x \in [0.5, 2.5]$ and one million verification vectors, enabling accuracy, latency, resource utilization, and energy to be compared under a common evaluation framework with architecture-specific timing constraints.

The results demonstrate substantial differences in the hardware and numerical characteristics of the evaluated architectures. The Direct LUT architecture provides the shortest pipelines, at the cost of the largest table-based memory requirement and the highest approximation error among the evaluated designs. LUT-based linear interpolation substantially reduces approximation error while retaining a relatively shallow pipeline. The two iterative architectures provide substantially higher numerical accuracy but require deeper pipelined datapaths and significantly greater DSP and flip-flop utilization. The Hybrid architecture combines a 9-bit-address interpolated seed with a single Newton-Raphson refinement iteration, providing an intermediate point in the accuracy, latency, DSP, and energy trade-off space.

The comparison also demonstrates that fixed-point format selection has a direct interaction with reciprocal architecture. Q8.24 generally preserves substantially higher numerical accuracy for the iterative and hybrid architectures because of its larger fractional field, whereas Q16.16 reduces fractional precision and consequently increases the observed approximation error for these architectures. At the same time, the change in fractional precision does not produce a uniform hardware advantage across all architectures, illustrating that arithmetic precision, memory organization, pipeline structure, and FPGA resource mapping must be considered together when selecting a fixed-point reciprocal implementation.

Across the evaluated designs, the maximum relative error spans more than three orders of magnitude, while latency spans more than an order of magnitude. Energy per operation varies by approximately 3.5 $\times$, from 0.346 nJ/op to 1.219 nJ/op. The three-objective Pareto analysis further shows that all five architectures are strictly non-dominated among the evaluated configurations within each fixed-point format: each occupies a distinct point in the accuracy-latency-energy design space rather than being uniformly superseded by another evaluated architecture. In Q16.16, however, this result depends on error differences of at most about 8% among the iterative and hybrid architectures, so it should be interpreted with that qualification. Reciprocal architecture selection is therefore application-dependent, with the appropriate implementation determined by the required balance among numerical precision, pipeline latency, throughput, energy, and FPGA resource availability.

Overall, the study provides a common evaluation framework for understanding these trade-offs across lookup-based, interpolated, iterative, and hybrid reciprocal architectures. Rather than identifying a universally optimal implementation, the results establish quantitative design points that can be used to select or further optimize a reciprocal architecture according to the constraints of a target FPGA application. Future work can extend this design space to additional FPGA families, alternative input distributions, vendor-supplied reciprocal/division IP, board-level power measurement, and systematic variation of seed-table resolution and iteration count to investigate configurations beyond the five evaluated architectures.

The complete RTL implementations, verification testbenches, datasets, and analysis scripts are publicly available in the accompanying GitHub repository [17], enabling reproducibility of the reported results.

Acknowledgments

Artificial intelligence (AI) tools like Claude and Gemini were used as research-assistance tools during the development and documentation of this work. The assistance included SystemVerilog debugging, examination of fixed-point arithmetic and pipeline behavior, identification of potential RTL implementation issues, creation of block diagrams (Figures 1 to 5), and limited refinement of technical writing. AI-generated suggestions were treated as candidate solutions and were not accepted without verification. Proposed RTL modifications were validated through compilation and cycle-accurate simulation, while hardware-related changes were subsequently evaluated through FPGA synthesis and implementation. Numerical-analysis procedures were cross-checked against independently computed reference values and the common verification dataset described in this work. AI tools were not used to generate, fabricate, or alter experimental measurements. The reported FPGA resource utilization, timing, latency, power, energy, and numerical-accuracy results were obtained from the corresponding simulation and FPGA implementation workflows.

References

- [1] S. F. Anderson, J. G. Earle, R. E. Goldschmidt, and D. M. Powers. 1967. The IBM System/360 Model 91: Floating-Point Execution Unit. *IBM Journal of Research and Development* 11, 1 (Jan. 1967), 34–53. doi:10.1147/rd.111.0034
- [2] R. Andraka. 1998. A Survey of CORDIC Algorithms for FPGA Based Computers. In *Proceedings of the 1998 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. 191–200. doi:10.1145/275107.275139
- [3] Jean-Louis Beuchat and Arnaud Tisserand. 2002. Small Multiplier-Based Multiplication and Division Operators for Virtex-II Devices. In *Proceedings of the 12th International Conference on Field-Programmable Logic and Applications (FPL)*. 513–522. doi:10.1007/3-540-46117-5_54
- [4] D. Chen, B. Zhou, Z. Guo, and P. Nilsson. 2005. Design and Implementation of Reciprocal Unit. In *Proceedings of the IEEE Midwest Symposium on Circuits and Systems (MWSCAS)*. 1318–1321. doi:10.1109/MWSCAS.2005.1594352
- [5] D. Das Sarma and D. W. Matula. 1995. Faithful Bipartite ROM Reciprocal Tables. In *Proceedings of the 12th IEEE Symposium on Computer Arithmetic (ARITH)*. 17–28. doi:10.1109/ARITH.1995.465381
- [6] Florent Dinechin and Arnaud Tisserand. 2005. Multipartite Table Methods. *IEEE Trans. Comput.* 54, 3 (March 2005), 319–330. doi:10.1109/TC.2005.54
- [7] Digilent, Inc. [n. d.]. Nexys A7 FPGA Board Reference Manual. Online. <https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual>
- [8] Milos D. Ercegovac, Laurent Imbert, David W. Matula, Jean-Michel Muller, and Guang Wei. 2000. Improving Goldschmidt Division, Square Root, and Square Root Reciprocal. *IEEE Trans. Comput.* 49, 7 (July 2000), 759–763. doi:10.1109/12.863046
- [9] Milos D. Ercegovac and Tomas Lang. 1994. *Division and Square Root: Digit-Recurrence Algorithms and Implementations*. Kluwer Academic Publishers, Boston, MA, USA.
- [10] Milos D. Ercegovac and Tomas Lang. 2004. *Digital Arithmetic*. Morgan Kaufmann, San Francisco, CA, USA.
- [11] Milos D. Ercegovac, Tomas Lang, Jean-Michel Muller, and Arnaud Tisserand. 2000. Reciprocation, Square Root, Inverse Square Root, and Some Elementary Functions Using Small Multipliers. *IEEE Trans. Comput.* 49, 7 (July 2000), 628–637. doi:10.1109/12.863031
- [12] Michael J. Flynn. 1970. On Division by Functional Iteration. *IEEE Trans. Comput.* C-19, 8 (Aug. 1970), 702–706. doi:10.1109/T-C.1970.223019
- [13] William L. Gallagher and Earl E. Swartzlander. 2000. Fault-Tolerant Newton-Raphson and Goldschmidt Dividers Using Time Shared TMR. *IEEE Trans. Comput.* 49, 6 (June 2000), 588–595. doi:10.1109/12.862218
- [14] Robert E. Goldschmidt. 1964. *Applications of Division by Convergence*. M.S. thesis. Massachusetts Institute of Technology, Cambridge, MA, USA.
- [15] Gonzalo Gutiérrez-Ramos, Ramón Parra-Michel, Eduardo Romero-Aguirre, Alberto Rodríguez-García, and Rodrigo Jaramillo-Ramírez. 2026. A Fast Fixed-Point Implementation for Division, Reciprocal, Square Root and Reciprocal Square Root Based on Newton-Raphson Method. *Electronics* 15, 13 (2026), 2899. doi:10.3390/electronics15132899
- [16] M. Ito, N. Takagi, and S. Yajima. 1997. Efficient Initial Approximation for Multiplicative Division and Square Root by a Multiplication With Operand Modification. *IEEE Trans. Comput.* 46, 4 (April 1997), 495–498. doi:10.1109/12.588066
- [17] Navaneeth K. [n. d.]. fixed-point-research. GitHub repository. <https://github.com/navaneethkutty/fixed-point-research>
- [18] M. F. Kasim, T. Adiono, M. Fahreza, and M. F. Zakiy. 2013. FPGA Implementation of Fixed-Point Divider Using Pre-Computed Values. *Procedia Technology* 11 (2013), 206–211. doi:10.1016/j.protcy.2013.12.182
- [19] I. Kong and E. E. Swartzlander. 2011. A Goldschmidt Division Method With Faster Than Quadratic Convergence. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 19, 4 (April 2011), 696–700. doi:10.1109/TVLSI.2009.2036926
- [20] U. Kucukkabak and A. Akkas. 2004. Design and Implementation of Reciprocal Unit Using Table Look-Up and Newton-Raphson Iteration. In *Proceedings of the Euromicro Symposium on Digital System Design (DSD)*. 249–253. doi:10.1109/DSD.2004.1333284
- [21] D. Kumar, P. Saha, and A. Dandapat. 2017. Hardware Implementation of Methodologies of Fixed Point Division Algorithms. *International Journal of Smart Sensing and Intelligent Systems* 10, 3 (2017), 1–16. doi:10.21307/ijssis-2017-227

- [22] Charles E. Leiserson and James B. Saxe. 1991. Retiming Synchronous Circuitry. *Algorithmica* 6, 1 (1991), 5–35. doi:10.1007/BF01759032
- [23] E. Libessart, M. Arzel, C. Lahuec, and F. P. Andriulli. 2017. A Scaling-Less Newton-Raphson Pipelined Implementation for a Fixed-Point Reciprocal Operator. *IEEE Signal Processing Letters* 24, 6 (June 2017), 789–793. doi:10.1109/LSP.2017.2694225
- [24] Michael Lunglmayr. 2021. Efficient Non-sequential Division for FPGAs. *arXiv preprint arXiv:2105.05747* (2021). doi:10.48550/arXiv.2105.05747
- [25] Jean-Michel Muller. 1997. *Elementary Functions: Algorithms and Implementation*. Birkhäuser, Boston, MA, USA. doi:10.1007/978-1-4757-2646-6
- [26] D. M. Muñoz, D. F. Sanchez, C. H. Llanos, and M. Ayala-Rincón. 2010. Tradeoff of FPGA Design of a Floating-Point Library for Arithmetic Operators. *Journal of Integrated Circuits and Systems* 5, 1 (2010), 42–52. doi:10.29292/jics.v5i1.309
- [27] Alberto Nannarelli and Tomas Lang. 1999. Low-Power Divider. *IEEE Trans. Comput.* 48, 1 (Jan. 1999), 2–14. doi:10.1109/12.802356
- [28] Stuart F. Oberman and Michael J. Flynn. 1997. Division Algorithms and Implementations. *IEEE Trans. Comput.* 46, 8 (Aug. 1997), 833–854. doi:10.1109/12.609274
- [29] G. Paim, P. Marques, E. A. C. da Costa, S. J. M. de Almeida, and S. Bampi. 2017. Improved Goldschmidt Algorithm for Fast and Energy-Efficient Fixed-Point Divider. In *Proceedings of the 24th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*. 482–485. doi:10.1109/ICECS.2017.8292070
- [30] Behrooz Parhami. 2000. *Computer Arithmetic: Algorithms and Hardware Designs*. Oxford University Press, New York, NY, USA.
- [31] Juan-Antonio Piñeiro and Jesús D. Bruguera. 2002. High-Speed Double-Precision Computation of Reciprocal, Division, Square Root, and Inverse Square Root. *IEEE Trans. Comput.* 51, 12 (Dec. 2002), 1377–1388. doi:10.1109/TC.2002.1146704
- [32] Juan-Antonio Piñeiro, Stuart F. Oberman, Jean-Michel Muller, and Jesús D. Bruguera. 2005. High-Speed Function Approximation Using a Minimax Quadratic Interpolator. *IEEE Trans. Comput.* 54, 3 (March 2005), 304–318. doi:10.1109/TC.2005.52
- [33] A. Rodríguez-García, L. Pizano-Escalante, R. Parra-Michel, O. Longoria-Gandara, and J. Cortez. 2013. Fast Fixed-Point Divider Based on Newton-Raphson Method and Piecewise Polynomial Approximation. In *Proceedings of the International Conference on Reconfigurable Computing and FPGAs (ReConFig)*. doi:10.1109/ReConFig.2013.6732291
- [34] Michael J. Schulte and James E. Stine. 1999. Approximating Elementary Functions with Symmetric Bipartite Tables. *IEEE Trans. Comput.* 48, 8 (Aug. 1999), 842–847. doi:10.1109/12.795125
- [35] Peter Soderquist and Miriam Leeser. 1996. Area and Performance Tradeoffs in Floating-Point Divide and Square-Root Implementations. *Comput. Surveys* 28, 3 (Sept. 1996), 518–564. doi:10.1145/243439.243481
- [36] N. Sorokin. 2006. Implementation of High-Speed Fixed-Point Dividers on FPGA. *Journal of Computer Science and Technology* 6, 1 (April 2006), 8–11.
- [37] J. E. Volder. 1959. The CORDIC Trigonometric Computing Technique. *IRE Transactions on Electronic Computers* EC-8, 3 (Sept. 1959), 330–334. doi:10.1109/TEC.1959.522693
- [38] X. Wang and B. Nelson. 2003. Tradeoffs of Designing Floating-Point Division and Square Root on Virtex FPGAs. In *Proceedings of the 11th Annual IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM)*. 195–203. doi:10.1109/FPGA.2003.1227255
- [39] Nicholas Weaver, Yuri Markovskiy, Yatish Patel, and John Wawrzynek. 2003. Post-Placement C-Slow Retiming for the Xilinx Virtex FPGA. In *Proceedings of the 11th ACM International Symposium on Field-Programmable Gate Arrays (FPGA)*. Monterey, CA, USA, 185–194. doi:10.1145/611817.611845
- [40] Xilinx, Inc. 2018. *7 Series FPGAs DSP48E1 Slice User Guide* (1.10 ed.). Xilinx, Inc. https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1

Received XX October 2026; revised XX October 2026; accepted XX January 2027